Threagile
Agile Threat Modeling

Threat Model Report
VaultNote Threat Model

18 April 2026

VaultNote Security Team

https://github.com/vaultnote

Table of Contents

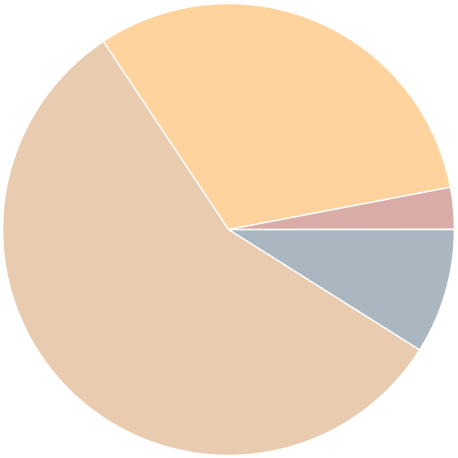
Management Summary

Threagile toolkit was used to model the architecture of "VaultNote Threat Model" and derive risks by analyzing the components and data flows. The risks identified during this analysis are shown in the following chapters. Identified risks during threat modeling do not necessarily mean that the vulnerability associated with this risk actually exists: it is more to be seen as a list of potential risks and threats, which should be individually reviewed and reduced by removing false positives. For the remaining risks it should be checked in the design and implementation of "VaultNote Threat Model" whether the mitigation advices have been applied or not.

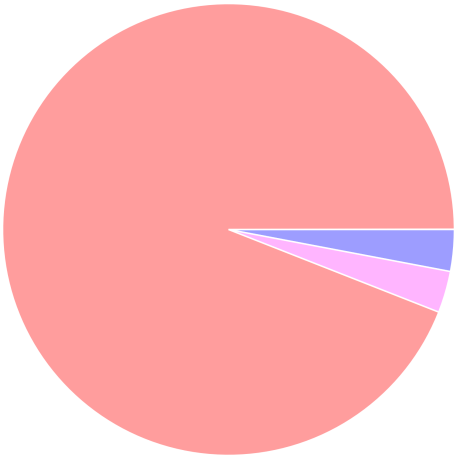
Each risk finding references a chapter of the OWASP ASVS (Application Security Verification Standard) audit checklist. The OWASP ASVS checklist should be considered as an inspiration by architects and developers to further harden the application in a Defense-in-Depth approach. Additionally, for each risk finding a link towards a matching OWASP Cheat Sheet or similar with technical details about how to implement a mitigation is given.

In total **67 initial risks** in **26 categories** have been identified during the threat modeling process:

0 critical risk
2 high risk
21 elevated risk
38 medium risk
6 low risk



63 unchecked
0 in discussion
2 accepted
2 in progress
0 *mitigated*
0 *false positive*



VaultNote is a containerized note-taking application comprised of a React SPA, Nginx reverse proxy, Node.js/Express REST API, PostgreSQL database, Redis session store, and MinIO object storage. The architecture deliberately contains several intentional misconfigurations (unencrypted internal HTTP, no encryption at rest, hardcoded credentials) to demonstrate automated threat detection via Threagile. All findings below are expected and serve as demonstration targets.

Impact Analysis of 67 Initial Risks in 26 Categories

The most prevalent impacts of the **67 initial risks** (distributed over **26 risk categories**) are (taking the severity ratings into account and using the highest for each category):
Risk finding paragraphs are clickable and link to the corresponding chapter.

High: **Asset Using Known Default or Hardcoded Credentials**: 1 Initial Risk - Exploitation likelihood is *Very Likely* with *High* impact.
Any attacker with network access can authenticate using default credentials, exfiltrate all stored data, modify records, or pivot into adjacent systems.

High: **SQL/NoSQL-Injection**: 2 Initial Risks - Exploitation likelihood is *Very Likely* with *High* impact.
If this risk is unmitigated, attackers might be able to modify SQL/NoSQL queries to steal and modify data and eventually further escalate towards a deeper system penetration via code executions.

Elevated: **Lateral Movement via Shared Runtime**: 1 Initial Risk - Exploitation likelihood is *Likely* with *High* impact.
An attacker who compromises any asset on the shared runtime may pivot to other co-located assets, including those in higher trust zones.

Elevated: **Missing Authentication**: 2 Initial Risks - Exploitation likelihood is *Likely* with *High* impact.
If this risk is unmitigated, attackers might be able to access or modify sensitive data in an unauthenticated way.

Elevated: **Missing Cloud Hardening**: 3 Initial Risks - Exploitation likelihood is *Unlikely* with *Very High* impact.
If this risk is unmitigated, attackers might access cloud components in an unintended way.

Elevated: **Missing Content-Security-Policy Header on Web Entry Point**: 1 Initial Risk - Exploitation likelihood is *Likely* with *Medium* impact.
An attacker who successfully injects client-side script can exfiltrate authentication tokens, impersonate users, and access all data the victim can access. Without CSP, there is no browser-enforced barrier against inline script execution or data exfiltration.

Elevated: **Missing Hardening**: 3 Initial Risks - Exploitation likelihood is *Likely* with *Medium* impact.
If this risk remains unmitigated, attackers might be able to easier attack high-value targets.

Elevated: **Path-Traversal**: 3 Initial Risks - Exploitation likelihood is *Very Likely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to read sensitive files (configuration data, key/credential files, deployment files, business data files, etc.) from the filesystem of affected components.

Elevated: **SQL/NoSQL-Injection**: 2 Initial Risks - Exploitation likelihood is *Very Likely* with *High* impact.
If this risk is unmitigated, attackers might be able to modify SQL/NoSQL queries to steal and modify data and eventually further escalate towards a deeper system penetration via code executions.

Elevated: **Server-Side Request Forgery (SSRF)**: 8 Initial Risks - Exploitation likelihood is *Likely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to access sensitive services or files of network-reachable components by modifying outgoing calls of affected components.

Elevated: **Unencrypted Communication**: 4 Initial Risks - Exploitation likelihood is *Likely* with *High* impact.
If this risk is unmitigated, network attackers might be able to to eavesdrop on unencrypted sensitive data sent between components.

Elevated: **Unguarded Access From Internet**: 5 Initial Risks - Exploitation likelihood is *Very Likely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to directly attack sensitive systems without any hardening components in-between due to them being directly exposed on the internet.

Elevated: **Unguarded Direct Datastore Access**: 3 Initial Risks - Exploitation likelihood is *Likely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to directly attack sensitive data stores without any protecting components in-between.

Medium: **Accidental Secret Leak**: 1 Initial Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers which have access to affected sourcecode repositories or artifact registries might find secrets accidentally checked-in.

Medium: **Code Backdooring**: 3 Initial Risks - Exploitation likelihood is *Unlikely* with *High* impact.
If this risk remains unmitigated, attackers might be able to execute code on and completely takeover production environments.

Medium: **Container Base Image Backdooring**: 7 Initial Risks - Exploitation likelihood is *Unlikely* with *High* impact.
If this risk is unmitigated, attackers might be able to deeply persist in the target system by executing code in deployed containers.

Medium: **Container Platform Escape**: 1 Initial Risk - Exploitation likelihood is *Unlikely* with *High* impact.
If this risk is unmitigated, attackers which have successfully compromised a container (via other vulnerabilities) might be able to deeply persist in the target system by executing code in many deployed containers and the container platform itself.

Medium: **Missing Build Infrastructure**: 1 Initial Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to exploit risks unseen in this threat model due to critical build infrastructure components missing in the model.

Medium: **Missing Cloud Hardening**: 3 Initial Risks - Exploitation likelihood is *Unlikely* with *Very High* impact.
If this risk is unmitigated, attackers might access cloud components in an unintended way.

Medium: **Missing Identity Store**: 1 Initial Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to exploit risks unseen in this threat model in the identity provider/store that is currently missing in the model.

Medium: **Missing Two-Factor Authentication (2FA)**: 1 Initial Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to access or modify highly sensitive data without strong authentication.

Medium: **Missing Vault (Secret Storage)**: 1 Initial Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to easier steal config secrets (like credentials, private keys, client certificates, etc.) once a vulnerability to access files is present and exploited.

Medium: **Missing Web Application Firewall (WAF)**: 1 Initial Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to apply standard attack pattern tests at great speed without any filtering.

Medium: **Mixed Targets on Shared Runtime**: 1 Initial Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers successfully attacking other components of the system might have an easy path towards more valuable targets, as they are running on the same shared runtime.

Medium: **Server-Side Request Forgery (SSRF)**: 8 Initial Risks - Exploitation likelihood is *Likely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to access sensitive services or files of network-reachable components by modifying outgoing calls of affected components.

Medium: **Unchecked Deployment**: 3 Initial Risks - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk remains unmitigated, vulnerabilities in custom-developed software or their dependencies might not be identified during continuous deployment cycles.

Medium: **Unencrypted Technical Assets**: 5 Initial Risks - Exploitation likelihood is *Unlikely* with *High* impact.
If this risk is unmitigated, attackers might be able to access unencrypted data when successfully compromising sensitive components.

Medium: **Unguarded Access From Internet**: 5 Initial Risks - Exploitation likelihood is *Very Likely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to directly attack sensitive systems without any hardening components in-between due to them being directly exposed on the internet.

Medium: **Unguarded Direct Datastore Access**: 3 Initial Risks - Exploitation likelihood is *Likely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to directly attack sensitive data stores without any protecting components in-between.

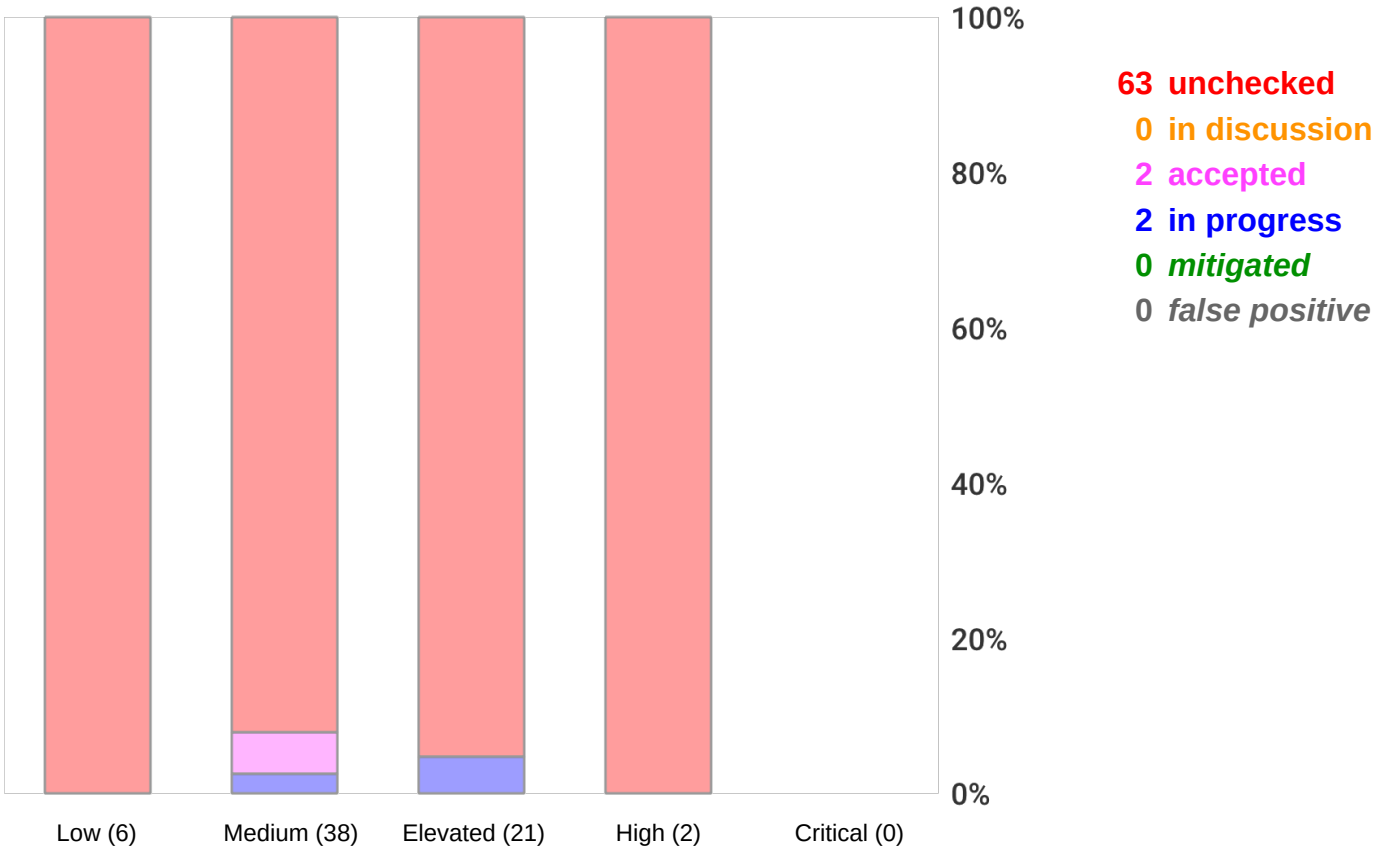
Low: **DoS-risky Access Across Trust-Boundary**: 3 Initial Risks - Exploitation likelihood is *Unlikely* with *Low* impact.
If this risk remains unmitigated, attackers might be able to disturb the availability of important parts of the system.

Low: **Unchecked Deployment**: 3 Initial Risks - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk remains unmitigated, vulnerabilities in custom-developed software or their dependencies might not be identified during continuous deployment cycles.

Low: **Unnecessary Technical Asset**: 2 Initial Risks - Exploitation likelihood is *Unlikely* with *Low* impact.
If this risk is unmitigated, attackers might be able to target unnecessary technical assets.

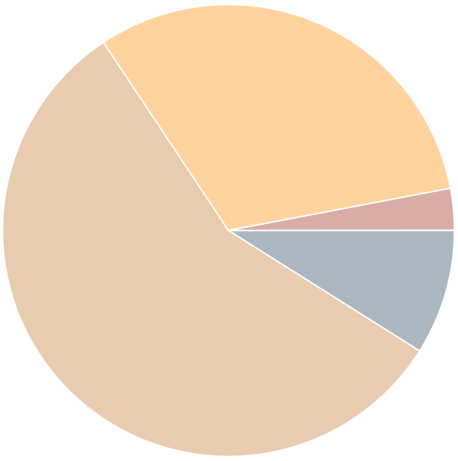
Risk Mitigation

The following chart gives a high-level overview of the risk tracking status (including mitigated risks):

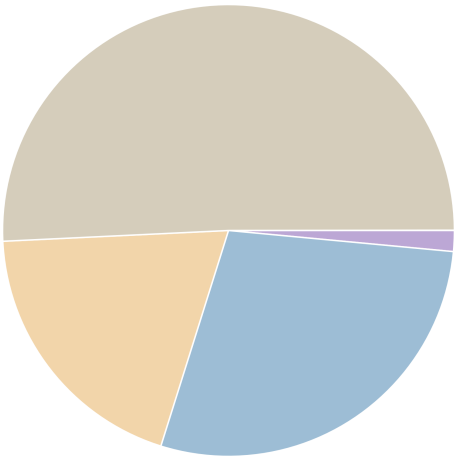


After removal of risks with status *mitigated* and *false positive* the following **67 remain unmitigated**:

0 unmitigated critical risk
2 unmitigated high risk
21 unmitigated elevated risk
38 unmitigated medium risk
6 unmitigated low risk



1 business side related
19 architecture related
13 development related
34 operations related



Asset Register

Technical Assets

API Server
Node.js/Express REST API. Handles authentication (JWT/bcrypt), CRUD on notes, file uploads to MinIO. Bridges the frontend and backend networks. Single most security-critical component — compromise equals full data access.

AWS API Gateway
AWS API Gateway (HTTP API) fronting the ECS API containers in production. Routes traffic from the ALB to backend ECS tasks. INTENTIONAL: No usage plan or throttling policy configured. No WAF association on the API gateway stage. Auth relies entirely on the API server (JWT) — no gateway-level auth enforcement.

AWS ECR Container Registry
AWS Elastic Container Registry storing VaultNote Docker images (api, nginx). INTENTIONAL: Image vulnerability scanning is disabled (ECR Basic scanning only, not Enhanced/Inspector). Images are not signed — no Cosign/Sigstore policy enforced. Repository is private but no admission controller validates signatures at deploy time.

AWS RDS PostgreSQL
AWS RDS PostgreSQL instance (production). Replaces the Docker-based PostgreSQL in production deployments. Automated backups enabled (7-day retention). INTENTIONAL: Encryption at rest is disabled (storage_encrypted = false in Terraform). Public accessibility is off but subnet group is in app subnet (not data subnet), creating potential exposure via compromised app instances.

AWS S3 Notes Bucket
AWS S3 bucket storing file attachments for production deployments. Replaces MinIO S3-compatible storage in the cloud environment. Bucket is private (Block Public Access enabled) but server-side encryption is not configured. INTENTIONAL: SSE-S3 or SSE-KMS not enabled. Objects are stored unencrypted at the storage layer — provider-side access exposes raw data.

AWS SES
AWS Simple Email Service used for transactional email delivery (note-share invitations, account verification). Managed service — not custom-developed. SES sending identity (domain) is verified; DKIM and SPF records are configured. No dedicated sending quota alarm; a compromised Lambda could exhaust the daily send limit and cause notification delivery failure.

Browser SPA
React Single Page Application served from Nginx. Users authenticate, manage notes, and upload files through this interface.

ECS Container Platform
AWS ECS (Fargate) running the API and Nginx containers in production. INTENTIONAL: ECR Enhanced Scanning (Inspector v2) is not enabled. Running containers are not scanned post-deployment for newly disclosed CVEs. Container images are pulled from ECR without

signature verification.

GitHub Actions Build Pipeline
GitHub Actions CI/CD pipeline. Builds Docker images, runs tests, pushes to ECR. INTENTIONAL: No SBOM generation (no Syft or CycloneDX step configured). No build provenance attestation (no SLSA provenance step). Hardcoded AWS_SECRET_ACCESS_KEY in workflow environment variables.

LLM Note Summarizer
OpenAI-compatible LLM inference endpoint used to generate AI summaries of user notes on demand. Deployed as a containerised model proxy (LiteLLM) with a public API endpoint for testing. INTENTIONAL: Endpoint is internet-accessible and lacks authentication — the has-authentication tag is intentionally absent to demonstrate the finding. No adversarial input shield (Llama Guard or guardrails) is configured. System prompt contains internal business context visible to adversarial probing.

Lambda Email Notifier
AWS Lambda function that sends transactional emails (note-share invitations, account verification) triggered by SQS messages from the API server. Processes user email addresses (PII) and note titles in notification payloads. INTENTIONAL: No dead letter queue (DLQ) configured on the SQS event source. Failed notifications are silently dropped after 3 retries with no audit trail.

MinIO Object Storage
S3-compatible object store for file attachments. INTENTIONAL MISCONFIGURATION: Uses default minioadmin/minioadmin credentials (visible in docker-compose.yml) and stores files without encryption at rest. HTTP-only (no TLS) within the Docker network.

Nginx Reverse Proxy
Nginx terminates TLS and forwards all API traffic to the Express server over plain HTTP (port 3000). This is the DMZ entry point.

Note RAG Pipeline
Retrieval-Augmented Generation pipeline that assembles context chunks from the vector store before forwarding to the LLM. Built on LangChain. INTENTIONAL: No input validation or injection guard on retrieved chunks (has-input-validation absent). A note containing prompt injection payloads could be retrieved as context and alter LLM behavior. No retrieval filtering — all results from the vector store are injected into context without relevance threshold or content sanitisation.

PostgreSQL Database
Primary relational datastore. Contains users table (email + bcrypt password hash), notes table (title + content), and attachments metadata. INTENTIONAL MISCONFIGURATION: No encryption at rest (encryption: none).

Redis Cache
In-memory store used as session store and JWT revocation list. INTENTIONAL MISCONFIGURATION: No encryption at rest. If Redis persistence is enabled and the RDB file is exfiltrated, session tokens are exposed.

VaultNote Source Repository
GitHub repository hosting the VaultNote monorepo (API, frontend, Terraform). Contains application secrets in compose files (minioadmin credentials visible in git history) and Terraform state references. INTENTIONAL: No branch protection rules configured — developers push directly to main, bypassing review and signed-commit requirements.

VaultNote Vector Store
Qdrant vector database storing embedding vectors for semantic search and RAG context retrieval. Vectors are generated by the embedding service from user note content. INTENTIONAL: No encryption at rest configured (encryption: none). The vector store is queryable by the LLM pipeline and the API server without row-level isolation — all user embeddings share the same collection. No access control list separates per-user embedding queries.

Data Assets

AI Model Responses
LLM-generated summaries and explanations of user notes. May contain paraphrased versions of sensitive note content. Cached temporarily in the API response layer.

Build Artifacts
Docker container images, compiled source bundles, and dependency lockfiles produced by the CI/CD pipeline. Integrity is critical — these are the deployable units of the application.

Embedding Vectors
High-dimensional vector representations of user note content generated by the embedding service. Semantically encodes note information even without storing the original text.

File Attachments
Binary file uploads attached to notes. May include documents, images, or other sensitive files depending on note usage.

Note Content
User-authored note text — may include personal, financial, or health information depending on how users employ the application.

Notification Payloads
Email notification payloads containing user email addresses and note titles sent via Lambda to AWS SES. Contains PII (email address) and indirect note content references.

Session Tokens
JWT tokens and Redis session keys used to maintain authenticated state. Theft allows session hijacking without knowing the user's password.

User Credentials
Email addresses and bcrypt-hashed passwords used for authentication. Compromise allows full account takeover.

Impact Analysis of 67 Remaining Risks in 26 Categories

The most prevalent impacts of the **67 remaining risks** (distributed over **26 risk categories**) are (taking the severity ratings into account and using the highest for each category):
Risk finding paragraphs are clickable and link to the corresponding chapter.

High: **Asset Using Known Default or Hardcoded Credentials**: 1 Remaining Risk - Exploitation likelihood is *Very Likely* with *High* impact.
Any attacker with network access can authenticate using default credentials, exfiltrate all stored data, modify records, or pivot into adjacent systems.

High: **SQL/NoSQL-Injection**: 2 Remaining Risks - Exploitation likelihood is *Very Likely* with *High* impact.
If this risk is unmitigated, attackers might be able to modify SQL/NoSQL queries to steal and modify data and eventually further escalate towards a deeper system penetration via code executions.

Elevated: **Lateral Movement via Shared Runtime**: 1 Remaining Risk - Exploitation likelihood is *Likely* with *High* impact.
An attacker who compromises any asset on the shared runtime may pivot to other co-located assets, including those in higher trust zones.

Elevated: **Missing Authentication**: 2 Remaining Risks - Exploitation likelihood is *Likely* with *High* impact.
If this risk is unmitigated, attackers might be able to access or modify sensitive data in an unauthenticated way.

Elevated: **Missing Cloud Hardening**: 3 Remaining Risks - Exploitation likelihood is *Unlikely* with *Very High* impact.
If this risk is unmitigated, attackers might access cloud components in an unintended way.

Elevated: **Missing Content-Security-Policy Header on Web Entry Point**: 1 Remaining Risk - Exploitation likelihood is *Likely* with *Medium* impact.
An attacker who successfully injects client-side script can exfiltrate authentication tokens, impersonate users, and access all data the victim can access. Without CSP, there is no browser-enforced barrier against inline script execution or data exfiltration.

Elevated: **Missing Hardening**: 3 Remaining Risks - Exploitation likelihood is *Likely* with *Medium* impact.
If this risk remains unmitigated, attackers might be able to easier attack high-value targets.

Elevated: **Path-Traversal**: 3 Remaining Risks - Exploitation likelihood is *Very Likely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to read sensitive files (configuration data, key/credential files, deployment files, business data files, etc.) from the filesystem of affected components.

Elevated: **SQL/NoSQL-Injection**: 2 Remaining Risks - Exploitation likelihood is *Very Likely* with *High* impact.
If this risk is unmitigated, attackers might be able to modify SQL/NoSQL queries to steal and modify data and eventually further escalate towards a deeper system penetration via code executions.

Elevated: **Server-Side Request Forgery (SSRF)**: 8 Remaining Risks - Exploitation likelihood is *Likely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to access sensitive services or files of network-reachable components by modifying outgoing calls of affected components.

Elevated: **Unencrypted Communication**: 4 Remaining Risks - Exploitation likelihood is *Likely* with *High* impact.
If this risk is unmitigated, network attackers might be able to to eavesdrop on unencrypted sensitive data sent between components.

Elevated: **Unguarded Access From Internet**: 5 Remaining Risks - Exploitation likelihood is *Very Likely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to directly attack sensitive systems without any hardening components in-between due to them being directly exposed on the internet.

Elevated: **Unguarded Direct Datastore Access**: 3 Remaining Risks - Exploitation likelihood is *Likely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to directly attack sensitive data stores without any protecting components in-between.

Medium: **Accidental Secret Leak**: 1 Remaining Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers which have access to affected sourcecode repositories or artifact registries might find secrets accidentally checked-in.

Medium: **Code Backdooring**: 3 Remaining Risks - Exploitation likelihood is *Unlikely* with *High* impact.
If this risk remains unmitigated, attackers might be able to execute code on and completely takeover production environments.

Medium: **Container Base Image Backdooring**: 7 Remaining Risks - Exploitation likelihood is *Unlikely* with *High* impact.
If this risk is unmitigated, attackers might be able to deeply persist in the target system by executing code in deployed containers.

Medium: **Container Platform Escape**: 1 Remaining Risk - Exploitation likelihood is *Unlikely* with *High* impact.
If this risk is unmitigated, attackers which have successfully compromised a container (via other vulnerabilities) might be able to deeply persist in the target system by executing code in many deployed containers and the container platform itself.

Medium: **Missing Build Infrastructure**: 1 Remaining Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to exploit risks unseen in this threat model due to critical build infrastructure components missing in the model.

Medium: **Missing Cloud Hardening**: 3 Remaining Risks - Exploitation likelihood is *Unlikely* with *Very High* impact.
If this risk is unmitigated, attackers might access cloud components in an unintended way.

Medium: **Missing Identity Store**: 1 Remaining Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to exploit risks unseen in this threat model in the identity provider/store that is currently missing in the model.

Medium: **Missing Two-Factor Authentication (2FA)**: 1 Remaining Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to access or modify highly sensitive data without strong authentication.

Medium: **Missing Vault (Secret Storage)**: 1 Remaining Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to easier steal config secrets (like credentials, private keys, client certificates, etc.) once a vulnerability to access files is present and exploited.

Medium: **Missing Web Application Firewall (WAF)**: 1 Remaining Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to apply standard attack pattern tests at great speed without any filtering.

Medium: **Mixed Targets on Shared Runtime**: 1 Remaining Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers successfully attacking other components of the system might have an easy path towards more valuable targets, as they are running on the same shared runtime.

Medium: **Server-Side Request Forgery (SSRF)**: 8 Remaining Risks - Exploitation likelihood is *Likely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to access sensitive services or files of network-reachable components by modifying outgoing calls of affected components.

Medium: **Unchecked Deployment**: 3 Remaining Risks - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk remains unmitigated, vulnerabilities in custom-developed software or their dependencies might not be identified during continuous deployment cycles.

Medium: **Unencrypted Technical Assets**: 5 Remaining Risks - Exploitation likelihood is *Unlikely* with *High* impact.
If this risk is unmitigated, attackers might be able to access unencrypted data when successfully compromising sensitive components.

Medium: **Unguarded Access From Internet**: 5 Remaining Risks - Exploitation likelihood is *Very Likely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to directly attack sensitive systems without any hardening components in-between due to them being directly exposed on the internet.

Medium: **Unguarded Direct Datastore Access**: 3 Remaining Risks - Exploitation likelihood is *Likely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to directly attack sensitive data stores without any protecting components in-between.

Low: **DoS-risky Access Across Trust-Boundary**: 3 Remaining Risks - Exploitation likelihood is *Unlikely* with *Low* impact.
If this risk remains unmitigated, attackers might be able to disturb the availability of important parts of the system.

Low: **Unchecked Deployment**: 3 Remaining Risks - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk remains unmitigated, vulnerabilities in custom-developed software or their dependencies might not be identified during continuous deployment cycles.

Low: **Unnecessary Technical Asset**: 2 Remaining Risks - Exploitation likelihood is *Unlikely* with *Low* impact.
If this risk is unmitigated, attackers might be able to target unnecessary technical assets.

Application Overview

Business Criticality

The overall business criticality of "VaultNote Threat Model" was rated as:

(archive | operational | **IMPORTANT** | critical | mission-critical)

Business Overview

VaultNote allows authenticated users to create, edit, and share encrypted notes with file attachments. The application stores sensitive user data and note content that must be protected against unauthorized disclosure.

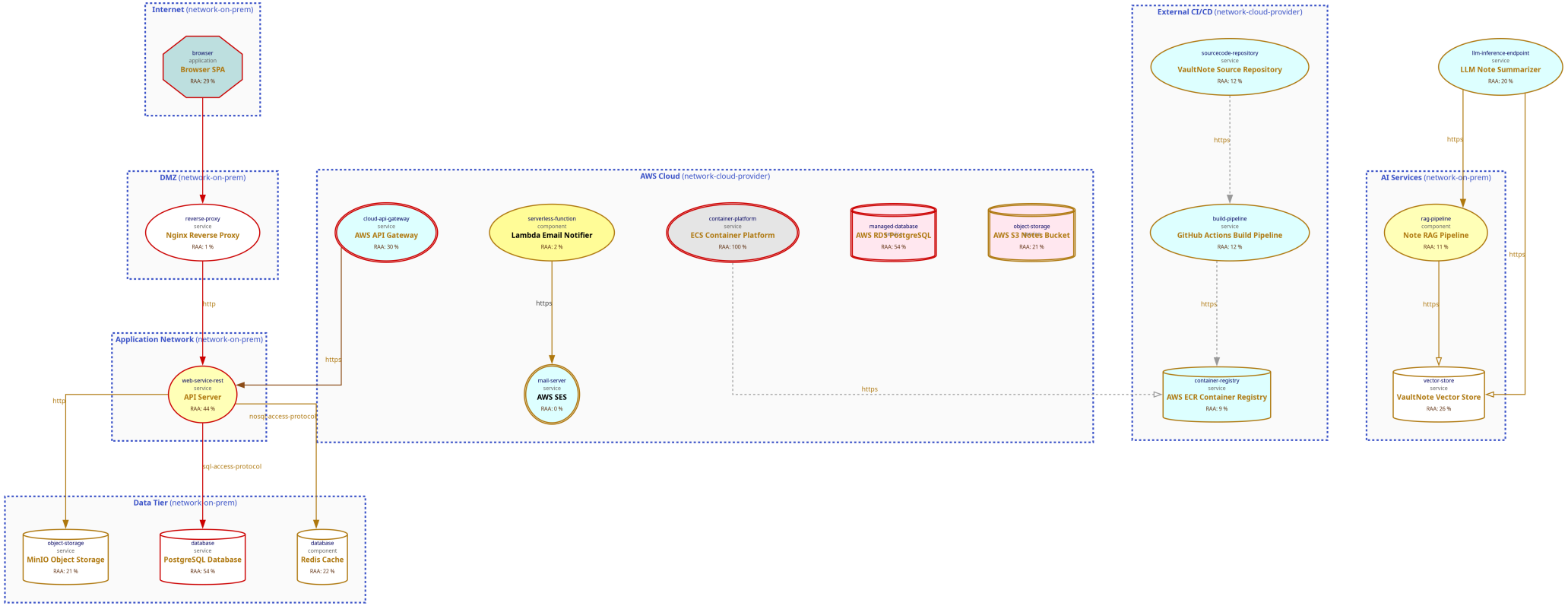
Technical Overview

Multi-tier containerized deployment: Nginx terminates TLS and proxies requests to the API over unencrypted HTTP. The API accesses PostgreSQL, Redis, and MinIO over plaintext TCP/HTTP within an internal Docker network.

Data-Flow Diagram

The following diagram was generated by Threagile based on the model input and gives a high-level overview of the data-flow between technical assets. The RAA value is the calculated *Relative Attacker Attractiveness* in percent. For a full high-resolution version of this diagram please refer to the PNG image file alongside this report.

Data-Flow Diagram - VaultNote Threat Model



Security Requirements

This chapter lists the custom security requirements which have been defined for the modeled target.

This list is not complete and regulatory or law relevant security requirements have to be taken into account as well. Also custom individual security requirements might exist for the project.

Abuse Cases

This chapter lists the custom abuse cases which have been defined for the modeled target.

This list is not complete and regulatory or law relevant abuse cases have to be taken into account as well. Also custom individual abuse cases might exist for the project.

Tag Listing

This chapter lists what tags are used by which elements.

ai
LLM Note Summarizer, Note RAG Pipeline, VaultNote Vector Store, AI Model Responses, Embedding Vectors, AI Services

api-gateway
AWS API Gateway

aws
AWS API Gateway, AWS ECR Container Registry, AWS RDS PostgreSQL, AWS S3 Notes Bucket, AWS SES, ECS Container Platform, Lambda Email Notifier, AWS Cloud

cache
Redis Cache

ci
GitHub Actions Build Pipeline

cloud-native
AWS API Gateway, AWS RDS PostgreSQL, AWS S3 Notes Bucket, AWS SES, ECS Container Platform, LLM Note Summarizer, Lambda Email Notifier, AWS Cloud

container-runtime
Docker Host

criticality-high
AWS RDS PostgreSQL, AWS S3 Notes Bucket, MinIO Object Storage, PostgreSQL Database, Redis Cache, VaultNote Vector Store

docker
Docker Host

ecr
AWS ECR Container Registry

ecs
ECS Container Platform

email
AWS SES

embeddings
VaultNote Vector Store, Embedding Vectors

express
API Server

fargate
ECS Container Platform

git
VaultNote Source Repository

github
VaultNote Source Repository

github-actions
GitHub Actions Build Pipeline

has-automated-backup
AWS RDS PostgreSQL

information-asset-container
AWS RDS PostgreSQL, AWS S3 Notes Bucket, MinIO Object Storage, PostgreSQL Database, Redis Cache, VaultNote Vector Store

intentional-misconfiguration
MinIO File Storage, MinIO Object Storage, HTTP to API Server

jwt
API Server

lambda
Lambda Email Notifier

langchain
Note RAG Pipeline

llm
LLM Note Summarizer

minio
MinIO Object Storage

nginx
Nginx Reverse Proxy

nodejs
API Server

notifications
Lambda Email Notifier

object-storage
MinIO Object Storage

openai
LLM Note Summarizer

pii
Notification Payloads

postgresql
AWS RDS PostgreSQL, PostgreSQL Database

qdrant
VaultNote Vector Store

rag
Note RAG Pipeline

rds
AWS RDS PostgreSQL

react
Browser SPA

redis
Redis Cache

relational
PostgreSQL Database

s3
AWS S3 Notes Bucket, MinIO Object Storage

ses
AWS SES

session-store
Redis Cache

spa
Browser SPA

supply-chain
AWS ECR Container Registry, GitHub Actions Build Pipeline, VaultNote Source Repository, Build Artifacts, External CI/CD

third-party-shared
AWS S3 Notes Bucket, MinIO Object Storage, PostgreSQL Database

tls-termination
Nginx Reverse Proxy

trike-actor
Browser SPA

trike-actor-untrusted
Browser SPA

trike-trust-level-admin
API Server

STRIDE Classification of Identified Risks

This chapter clusters and classifies the risks by STRIDE categories: In total **67 potential risks** have been identified during the threat modeling process of which **1 in the Spoofing** category, **23 in the Tampering** category, **0 in the Repudiation** category, **24 in the Information Disclosure** category, **3 in the Denial of Service** category, and **15 in the Elevation of Privilege** category.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Spoofing

Medium: **Missing Identity Store**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

The modeled architecture does not contain an identity store, which might be the risk of a model missing critical assets (and thus not seeing their risks).

Tampering

High: **SQL/NoSQL-Injection**: 2 / 2 Risks - Exploitation likelihood is *Very Likely* with *High* impact.

When a database is accessed via database access protocols SQL/NoSQL-Injection risks might arise. The risk rating depends on the sensitivity technical asset itself and of the data assets processed.

Elevated: **Missing Cloud Hardening**: 3 / 3 Risks - Exploitation likelihood is *Unlikely* with *Very High* impact.

Cloud components should be hardened according to the cloud vendor best practices. This affects their configuration, auditing, and further areas.

Elevated: **Missing Hardening**: 3 / 3 Risks - Exploitation likelihood is *Likely* with *Medium* impact.

Technical assets with a Relative Attacker Attractiveness (RAA) value of 55 % or higher should be explicitly hardened taking best practices and vendor hardening guides into account.

Elevated: **SQL/NoSQL-Injection**: 2 / 2 Risks - Exploitation likelihood is *Very Likely* with *High* impact.

When a database is accessed via database access protocols SQL/NoSQL-Injection risks might arise. The risk rating depends on the sensitivity technical asset itself and of the data assets processed.

Medium: **Code Backdooring**: 3 / 3 Risks - Exploitation likelihood is *Unlikely* with *High* impact.

For each build pipeline component Code Backdooring risks might arise where attackers compromise the build pipeline in order to let backdoored artifacts be shipped into production. Aside from direct code backdooring this includes backdooring of dependencies and even of more lower-level build infrastructure, like backdooring compilers (similar to what the XcodeGhost malware did) or dependencies.

Medium: **Container Base Image Backdooring**: 7 / 7 Risks - Exploitation likelihood is *Unlikely* with *High* impact.
When a technical asset is built using container technologies, Base Image Backdooring risks might arise where base images and other layers used contain vulnerable components or backdoors.

Medium: **Missing Build Infrastructure**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
The modeled architecture does not contain a build infrastructure (devops-client, sourcecode-repo, build-pipeline, etc.), which might be the risk of a model missing critical assets (and thus not seeing their risks). If the architecture contains custom-developed parts, the pipeline where code gets developed and built needs to be part of the model.

Medium: **Missing Cloud Hardening**: 3 / 3 Risks - Exploitation likelihood is *Unlikely* with *Very High* impact.
Cloud components should be hardened according to the cloud vendor best practices. This affects their configuration, auditing, and further areas.

Medium: **Missing Web Application Firewall (WAF)**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
To have a first line of filtering defense, security architectures with web-services or web-applications should include a WAF in front of them. Even though a WAF is not a replacement for security (all components must be secure even without a WAF) it adds another layer of defense to the overall system by delaying some attacks and having easier attack alerting through it.

Medium: **Unchecked Deployment**: 3 / 3 Risks - Exploitation likelihood is *Unlikely* with *Medium* impact.
For each build-pipeline component Unchecked Deployment risks might arise when the build-pipeline does not include established DevSecOps best-practices. DevSecOps best-practices scan as part of CI/CD pipelines for vulnerabilities in source- or byte-code, dependencies, container layers, and dynamically against running test systems. There are several open-source and commercial tools existing in the categories DAST, SAST, and IAST.

Low: **Unchecked Deployment**: 3 / 3 Risks - Exploitation likelihood is *Unlikely* with *Medium* impact.
For each build-pipeline component Unchecked Deployment risks might arise when the build-pipeline does not include established DevSecOps best-practices. DevSecOps best-practices scan as part of CI/CD pipelines for vulnerabilities in source- or byte-code, dependencies, container layers, and dynamically against running test systems. There are several open-source and commercial tools existing in the categories DAST, SAST, and IAST.

Repudiation

n/a

Information Disclosure

High: **Asset Using Known Default or Hardcoded Credentials**: 1 / 1 Risk - Exploitation likelihood is *Very Likely* with *High* impact.
Technical assets tagged as intentional-misconfiguration that store confidential data represent an immediately exploitable attack vector. Default credentials (e.g. minioadmin/minioadmin, admin/admin, postgres/postgres) are published in vendor documentation and are the first credentials an attacker tries after reaching the service network.

Elevated: **Missing Content-Security-Policy Header on Web Entry Point**: 1 / 1 Risk - Exploitation likelihood is *Likely* with *Medium* impact.
Web entry points (reverse proxies and web applications) that serve browser-based clients and process authentication tokens without a Content-Security-Policy (CSP) header are vulnerable to XSS-facilitated token theft. An attacker who achieves XSS in the browser can read localStorage/sessionStorage tokens and replay them without any client-side protection if CSP is absent.

Elevated: **Path-Traversal**: 3 / 3 Risks - Exploitation likelihood is *Very Likely* with *Medium* impact.
When a filesystem is accessed Path-Traversal or Local-File-Inclusion (LFI) risks might arise. The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed.

Elevated: **Server-Side Request Forgery (SSRF)**: 8 / 8 Risks - Exploitation likelihood is *Likely* with *Medium* impact.
When a server system (i.e. not a client) is accessing other server systems via typical web protocols Server-Side Request Forgery (SSRF) or Local-File-Inclusion (LFI) or Remote-File-Inclusion (RFI) risks might arise.

Elevated: **Unencrypted Communication**: 4 / 4 Risks - Exploitation likelihood is *Likely* with *High* impact.
Due to the confidentiality and/or integrity rating of the data assets transferred over the communication link this connection must be encrypted.

Medium: **Accidental Secret Leak**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
Sourcecode repositories (including their histories) as well as artifact registries can accidentally contain secrets like checked-in or packaged-in passwords, API tokens, certificates, crypto keys, etc.

Medium: **Missing Vault (Secret Storage)**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
In order to avoid the risk of secret leakage via config files (when attacked through vulnerabilities being able to read files like Path-Traversal and others), it is best practice to use a separate hardened process with proper authentication, authorization, and audit logging to access config secrets (like credentials, private keys, client certificates, etc.). This component is usually some kind of Vault.

Medium: **Server-Side Request Forgery (SSRF)**: 8 / 8 Risks - Exploitation likelihood is *Likely* with *Medium* impact.
When a server system (i.e. not a client) is accessing other server systems via typical web protocols Server-Side Request Forgery (SSRF) or Local-File-Inclusion (LFI) or Remote-File-Inclusion (RFI) risks might arise.

Medium: **Unencrypted Technical Assets**: 5 / 5 Risks - Exploitation likelihood is *Unlikely* with *High* impact.
Due to the confidentiality rating of the technical asset itself and/or the stored data assets this technical asset must be encrypted. The risk rating depends on the sensitivity technical asset itself and of the data assets stored.

Denial of Service

Low: **DoS-risky Access Across Trust-Boundary**: 3 / 3 Risks - Exploitation likelihood is *Unlikely* with *Low* impact.
Assets accessed across trust boundaries with critical or mission-critical availability rating are more prone to Denial-of-Service (DoS) risks.

Elevation of Privilege

Elevated: **Missing Authentication**: 2 / 2 Risks - Exploitation likelihood is *Likely* with *High* impact.
Technical assets (especially multi-tenant systems) should authenticate incoming requests when the asset processes sensitive data.

Elevated: **Unguarded Access From Internet**: 5 / 5 Risks - Exploitation likelihood is *Very Likely* with *Medium* impact.
Internet-exposed assets must be guarded by a protecting service, application, or reverse-proxy.

Elevated: **Unguarded Direct Datastore Access**: 3 / 3 Risks - Exploitation likelihood is *Likely* with *Medium* impact.
Data stores accessed across trust boundaries must be guarded by some protecting service or application.

Medium: **Container Platform Escape**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *High* impact.
Container platforms are especially interesting targets for attackers as they host big parts of a containerized runtime infrastructure. When not configured and operated with security best practices in mind, attackers might exploit a vulnerability inside an container and escape towards the platform as highly privileged users. These scenarios might give attackers capabilities to attack every other container as owning the container platform (via container escape attacks) equals to owning every container.

Medium: **Missing Two-Factor Authentication (2FA)**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
Technical assets (especially multi-tenant systems) should authenticate incoming requests with two-factor (2FA) authentication when the asset processes or stores highly sensitive data (in terms of confidentiality, integrity, and availability) and is accessed by humans.

Medium: **Mixed Targets on Shared Runtime**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
Different attacker targets (like frontend and backend/datastore components) should not be running on the same shared (underlying) runtime.

Medium: **Unguarded Access From Internet**: 5 / 5 Risks - Exploitation likelihood is *Very Likely* with *Medium* impact.
Internet-exposed assets must be guarded by a protecting service, application, or reverse-proxy.

Medium: **Unguarded Direct Datastore Access**: 3 / 3 Risks - Exploitation likelihood is *Likely* with *Medium* impact.
Data stores accessed across trust boundaries must be guarded by some protecting service or application.

Low: **Unnecessary Technical Asset**: 2 / 2 Risks - Exploitation likelihood is *Unlikely* with *Low* impact.
When a technical asset does not process any data assets, this is an indicator for an unnecessary technical asset (or for an incomplete model). This is also the case if the asset has no communication links (either outgoing or incoming).

Assignment by Function

This chapter clusters and assigns the risks by functions which are most likely able to check and mitigate them: In total **67 potential risks** have been identified during the threat modeling process of which **1 should be checked by Business Side**, **19 should be checked by Architecture**, **13 should be checked by Development**, and **34 should be checked by Operations**.
Risk finding paragraphs are clickable and link to the corresponding chapter.

Business Side

Medium: **Missing Two-Factor Authentication (2FA)**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
Apply an authentication method to the technical asset protecting highly sensitive data via two-factor authentication for human users.

Architecture

Elevated: **Lateral Movement via Shared Runtime**: 1 / 1 Risk - Exploitation likelihood is *Likely* with *High* impact.
Separate assets from different trust zones onto dedicated runtimes (separate node pools, dedicated VMs, distinct container namespaces with NetworkPolicy). Apply seccomp/AppArmor profiles and disable privilege escalation.

Elevated: **Missing Authentication**: 2 / 2 Risks - Exploitation likelihood is *Likely* with *High* impact.
Apply an authentication method to the technical asset. To protect highly sensitive data consider the use of two-factor authentication for human users.

Elevated: **Unguarded Access From Internet**: 5 / 5 Risks - Exploitation likelihood is *Very Likely* with *Medium* impact.
Encapsulate the asset behind a guarding service, application, or reverse-proxy. For admin maintenance a bastion-host should be used as a jump-server. For file transfer a store-and-forward-host should be used as an indirect file exchange platform.

Elevated: **Unguarded Direct Datastore Access**: 3 / 3 Risks - Exploitation likelihood is *Likely* with *Medium* impact.
Encapsulate the datastore access behind a guarding service or application.

Medium: **Missing Build Infrastructure**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
Include the build infrastructure in the model.

Medium: **Missing Identity Store**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
Include an identity store in the model if the application has a login.

Medium: **Missing Vault (Secret Storage)**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
Consider using a Vault (Secret Storage) to securely store and access config secrets (like credentials, private keys, client certificates, etc.).

Medium: **Unchecked Deployment**: 3 / 3 Risks - Exploitation likelihood is *Unlikely* with *Medium* impact.
Apply DevSecOps best-practices and use scanning tools to identify vulnerabilities in source- or byte-code,dependencies, container layers, and optionally also via dynamic scans against running test systems.

Medium: **Unguarded Access From Internet**: 5 / 5 Risks - Exploitation likelihood is *Very Likely* with *Medium* impact.
Encapsulate the asset behind a guarding service, application, or reverse-proxy. For admin maintenance a bastion-host should be used as a jump-server. For file transfer a store-and-forward-host should be used as an indirect file exchange platform.

Medium: **Unguarded Direct Datastore Access**: 3 / 3 Risks - Exploitation likelihood is *Likely* with *Medium* impact.
Encapsulate the datastore access behind a guarding service or application.

Low: **Unchecked Deployment**: 3 / 3 Risks - Exploitation likelihood is *Unlikely* with *Medium* impact.
Apply DevSecOps best-practices and use scanning tools to identify vulnerabilities in source- or byte-code,dependencies, container layers, and optionally also via dynamic scans against running test systems.

Low: **Unnecessary Technical Asset**: 2 / 2 Risks - Exploitation likelihood is *Unlikely* with *Low* impact.
Try to avoid using technical assets that do not process or store anything.

Development

High: **SQL/NoSQL-Injection**: 2 / 2 Risks - Exploitation likelihood is *Very Likely* with *High* impact.
Try to use parameter binding to be safe from injection vulnerabilities. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

Elevated: **Path-Traversal**: 3 / 3 Risks - Exploitation likelihood is *Very Likely* with *Medium* impact.
Before accessing the file cross-check that it resides in the expected folder and is of the expected type and filename/suffix. Try to use a mapping if possible instead of directly accessing by a filename which is (partly or fully) provided by the caller. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

Elevated: **SQL/NoSQL-Injection**: 2 / 2 Risks - Exploitation likelihood is *Very Likely* with *High* impact.
Try to use parameter binding to be safe from injection vulnerabilities. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

Elevated: **Server-Side Request Forgery (SSRF)**: 8 / 8 Risks - Exploitation likelihood is *Likely* with *Medium* impact.
Try to avoid constructing the outgoing target URL with caller controllable values. Alternatively use a mapping (whitelist) when accessing outgoing URLs instead of creating them including caller controllable values. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

Medium: **Server-Side Request Forgery (SSRF)**: 8 / 8 Risks - Exploitation likelihood is *Likely* with *Medium* impact.
Try to avoid constructing the outgoing target URL with caller controllable values. Alternatively use a mapping (whitelist) when accessing outgoing URLs instead of creating them including caller controllable values. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

Operations

High: **Asset Using Known Default or Hardcoded Credentials**: 1 / 1 Risk - Exploitation likelihood is *Very Likely* with *High* impact.
Replace all default credentials with randomly generated secrets (minimum 32 chars). Store in a secrets manager (Vault, AWS Secrets Manager, etc.) rather than environment variables or compose files. Add credential rotation to the pre-deployment checklist.

Elevated: **Missing Cloud Hardening**: 3 / 3 Risks - Exploitation likelihood is *Unlikely* with *Very High* impact.
Apply hardening of all cloud components and services, taking special care to follow the individual risk descriptions (which depend on the cloud provider tags in the model).

Elevated: **Missing Content-Security-Policy Header on Web Entry Point**: 1 / 1 Risk - Exploitation likelihood is *Likely* with *Medium* impact.
Add a strict Content-Security-Policy header to all responses served to browser clients. At minimum use: default-src 'self'; script-src 'self'; object-src 'none'; base-uri 'none'. Combine with Subresource Integrity (SRI) for external scripts and avoid inline event handlers. Once deployed, tag the asset has-csp to suppress this finding.

Elevated: **Missing Hardening**: 3 / 3 Risks - Exploitation likelihood is *Likely* with *Medium* impact.
Try to apply all hardening best practices (like CIS benchmarks, OWASP recommendations, vendor recommendations, DevSec Hardening Framework, DBSAT for Oracle databases, and others).

Elevated: **Unencrypted Communication**: 4 / 4 Risks - Exploitation likelihood is *Likely* with *High* impact.
Apply transport layer encryption to the communication link.

Medium: **Accidental Secret Leak**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
Establish measures preventing accidental check-in or package-in of secrets into sourcecode repositories and artifact registries. This starts by using good .gitignore and .dockerignore files, but does not stop there. See for example tools like *\"git-secrets\"* or *\"Talisman\"* to have check-in preventive measures for secrets. Consider also to regularly scan your repositories for secrets accidentally checked-in using scanning tools like *"gitleaks"* or *"gitrob"*.

Medium: **Code Backdooring**: 3 / 3 Risks - Exploitation likelihood is *Unlikely* with *High* impact.
Reduce the attack surface of backdooring the build pipeline by not directly exposing the build pipeline components on the public internet and also not exposing it in front of unmanaged (out-of-scope) developer clients.Also consider the use of code signing to prevent code modifications.

Medium: **Container Base Image Backdooring**: 7 / 7 Risks - Exploitation likelihood is *Unlikely* with *High* impact.
Apply hardening of all container infrastructures (see for example the *CIS-Benchmarks for Docker and Kubernetes* and the *Docker Bench for Security*). Use only trusted base images of the original vendors, verify digital signatures and apply image creation best practices. Also consider using Google's *Distroless* base images or otherwise very small base images. Regularly execute container image scans with tools checking the layers for vulnerable components.

Medium: **Container Platform Escape**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *High* impact.
Apply hardening of all container infrastructures.

Medium: **Missing Cloud Hardening**: 3 / 3 Risks - Exploitation likelihood is *Unlikely* with *Very High* impact.
Apply hardening of all cloud components and services, taking special care to follow the individual risk descriptions (which depend on the cloud provider tags in the model).

Medium: **Missing Web Application Firewall (WAF)**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
Consider placing a Web Application Firewall (WAF) in front of the web-services and/or web-applications. For cloud environments many cloud providers offer pre-configured WAFs. Even reverse proxies can be enhances by a WAF component via ModSecurity plugins.

Medium: **Mixed Targets on Shared Runtime**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
Use separate runtime environments for running different target components or apply similar separation styles to prevent load- or breach-related problems originating from one more attacker-facing asset impacts also the other more critical rated backend/datastore assets.

Medium: **Unencrypted Technical Assets**: 5 / 5 Risks - Exploitation likelihood is *Unlikely* with *High* impact.
Apply encryption to the technical asset.

Low: **DoS-risky Access Across Trust-Boundary**: 3 / 3 Risks - Exploitation likelihood is *Unlikely* with *Low* impact.
Apply anti-DoS techniques like throttling and/or per-client load blocking with quotas. Also for maintenance access routes consider applying a VPN instead of public reachable interfaces. Generally applying redundancy on the targeted technical asset reduces the risk of DoS.

RAA Analysis

For each technical asset the **"Relative Attacker Attractiveness"** (RAA) value was calculated in percent. The higher the RAA, the more interesting it is for an attacker to compromise the asset. The calculation algorithm takes the sensitivity ratings and quantities of stored and processed data into account as well as the communication links of the technical asset. Neighbouring assets to high-value RAA targets might receive an increase in their RAA value when they have a communication link towards that target ("Pivoting-Factor").

The following lists all technical assets sorted by their RAA value from highest (most attacker attractive) to lowest. This list can be used to prioritize on efforts relevant for the most attacker-attractive technical assets:

Technical asset paragraphs are clickable and link to the corresponding chapter.

ECS Container Platform: RAA 100%
AWS ECS (Fargate) running the API and Nginx containers in production. INTENTIONAL: ECR Enhanced Scanning (Inspector v2) is not enabled. Running containers are not scanned post-deployment for newly disclosed CVEs. Container images are pulled from ECR without signature verification.

AWS RDS PostgreSQL: RAA 54%
AWS RDS PostgreSQL instance (production). Replaces the Docker-based PostgreSQL in production deployments. Automated backups enabled (7-day retention). INTENTIONAL: Encryption at rest is disabled (storage_encrypted = false in Terraform). Public accessibility is off but subnet group is in app subnet (not data subnet), creating potential exposure via compromised app instances.

PostgreSQL Database: RAA 54%
Primary relational datastore. Contains users table (email + bcrypt password hash), notes table (title + content), and attachments metadata. INTENTIONAL MISCONFIGURATION: No encryption at rest (encryption: none).

API Server: RAA 44%
Node.js/Express REST API. Handles authentication (JWT/bcrypt), CRUD on notes, file uploads to MinIO. Bridges the frontend and backend networks. Single most security-critical component — compromise equals full data access.

AWS API Gateway: RAA 30%
AWS API Gateway (HTTP API) fronting the ECS API containers in production. Routes traffic from the ALB to backend ECS tasks. INTENTIONAL: No usage plan or throttling policy configured. No WAF association on the API gateway stage. Auth relies entirely on the API server (JWT) — no gateway-level auth enforcement.

Browser SPA: RAA 29%
React Single Page Application served from Nginx. Users authenticate, manage notes, and upload files through this interface.

VaultNote Vector Store: RAA 26%
Qdrant vector database storing embedding vectors for semantic search and RAG context retrieval. Vectors are generated by the embedding service from user note content. INTENTIONAL: No encryption at rest configured (encryption: none). The vector store is queryable by the LLM pipeline and the API server without row-level isolation — all user embeddings share the same collection. No access control list separates per-user embedding queries.

Redis Cache: RAA 22%
In-memory store used as session store and JWT revocation list. INTENTIONAL MISCONFIGURATION: No encryption at rest. If Redis persistence is enabled and the RDB file is exfiltrated, session tokens are exposed.

AWS S3 Notes Bucket: RAA 21%
AWS S3 bucket storing file attachments for production deployments. Replaces MinIO S3-compatible storage in the cloud environment. Bucket is private (Block Public Access enabled) but server-side encryption is not configured. INTENTIONAL: SSE-S3 or SSE-KMS not enabled. Objects are stored unencrypted at the storage layer — provider-side access exposes raw data.

MinIO Object Storage: RAA 21%
S3-compatible object store for file attachments. INTENTIONAL MISCONFIGURATION: Uses default minioadmin/minioadmin credentials (visible in docker-compose.yml) and stores files without encryption at rest. HTTP-only (no TLS) within the Docker network.

LLM Note Summarizer: RAA 20%
OpenAI-compatible LLM inference endpoint used to generate AI summaries of user notes on demand. Deployed as a containerised model proxy (LiteLLM) with a public API endpoint for testing. INTENTIONAL: Endpoint is internet-accessible and lacks authentication — the has-authentication tag is intentionally absent to demonstrate the finding. No adversarial input shield (Llama Guard or guardrails) is configured. System prompt contains internal business context visible to adversarial probing.

GitHub Actions Build Pipeline: RAA 12%
GitHub Actions CI/CD pipeline. Builds Docker images, runs tests, pushes to ECR. INTENTIONAL: No SBOM generation (no Syft or CycloneDX step configured). No build provenance attestation (no SLSA provenance step). Hardcoded AWS_SECRET_ACCESS_KEY in workflow environment variables.

VaultNote Source Repository: RAA 12%
GitHub repository hosting the VaultNote monorepo (API, frontend, Terraform). Contains application secrets in compose files (minioadmin credentials visible in git history) and Terraform state references. INTENTIONAL: No branch protection rules configured — developers push directly to main, bypassing review and signed-commit requirements.

Note RAG Pipeline: RAA 11%
Retrieval-Augmented Generation pipeline that assembles context chunks from the vector store before forwarding to the LLM. Built on LangChain. INTENTIONAL: No input validation or injection guard on retrieved chunks (has-input-validation absent). A note containing prompt injection payloads could be retrieved as context and alter LLM behavior. No retrieval filtering — all results from the

vector store are injected into context without relevance threshold or content sanitisation.

AWS ECR Container Registry: RAA 9%
AWS Elastic Container Registry storing VaultNote Docker images (api, nginx). INTENTIONAL: Image vulnerability scanning is disabled (ECR Basic scanning only, not Enhanced/Inspector). Images are not signed — no Cosign/Sigstore policy enforced. Repository is private but no admission controller validates signatures at deploy time.

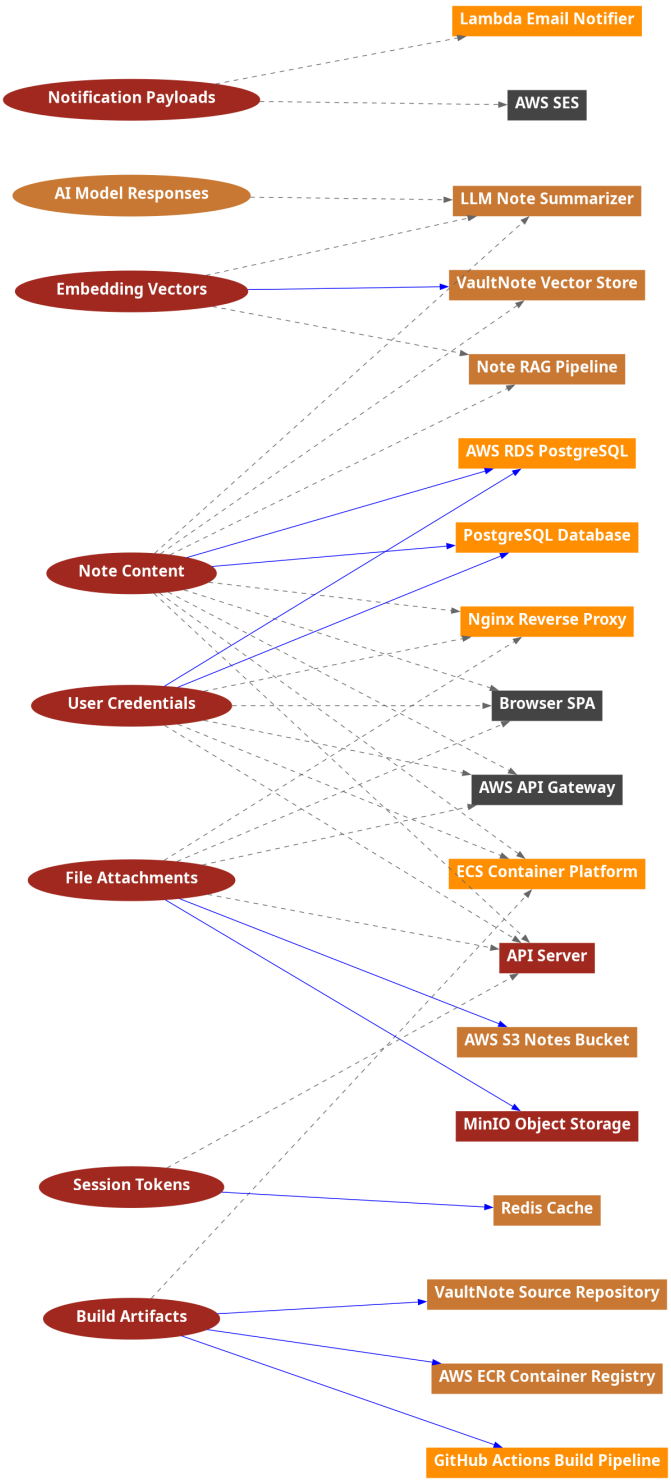
Lambda Email Notifier: RAA 2%
AWS Lambda function that sends transactional emails (note-share invitations, account verification) triggered by SQS messages from the API server. Processes user email addresses (PII) and note titles in notification payloads. INTENTIONAL: No dead letter queue (DLQ) configured on the SQS event source. Failed notifications are silently dropped after 3 retries with no audit trail.

Nginx Reverse Proxy: RAA 1%
Nginx terminates TLS and forwards all API traffic to the Express server over plain HTTP (port 3000). This is the DMZ entry point.

AWS SES: RAA 0%
AWS Simple Email Service used for transactional email delivery (note-share invitations, account verification). Managed service — not custom-developed. SES sending identity (domain) is verified; DKIM and SPF records are configured. No dedicated sending quota alarm; a compromised Lambda could exhaust the daily send limit and cause notification delivery failure.

Data Mapping

The following diagram was generated by Threagile based on the model input and gives a high-level distribution of data assets across technical assets. The color matches the identified data breach probability and risk level (see the "Data Breach Probabilities" chapter for more details). A solid line stands for *data is stored by the asset* and a dashed one means *data is processed by the asset*. For a full high-resolution version of this diagram please refer to the PNG image file alongside this report.



Out-of-Scope Assets: 0 Assets

This chapter lists all technical assets that have been defined as out-of-scope. Each one should be checked in the model whether it should better be included in the overall risk analysis:

Technical asset paragraphs are clickable and link to the corresponding chapter.

No technical assets have been defined as out-of-scope.

Potential Model Failures: 5 / 5 Risks

This chapter lists potential model failures where not all relevant assets have been modeled or the model might itself contain inconsistencies. Each potential model failure should be checked in the model against the architecture design:

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium: **Missing Build Infrastructure**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

The modeled architecture does not contain a build infrastructure (devops-client, sourcecode-repo, build-pipeline, etc.), which might be the risk of a model missing critical assets (and thus not seeing their risks). If the architecture contains custom-developed parts, the pipeline where code gets developed and built needs to be part of the model.

Medium: **Missing Identity Store**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

The modeled architecture does not contain an identity store, which might be the risk of a model missing critical assets (and thus not seeing their risks).

Medium: **Missing Vault (Secret Storage)**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

In order to avoid the risk of secret leakage via config files (when attacked through vulnerabilities being able to read files like Path-Traversal and others), it is best practice to use a separate hardened process with proper authentication, authorization, and audit logging to access config secrets (like credentials, private keys, client certificates, etc.). This component is usually some kind of Vault.

Low: **Unnecessary Technical Asset**: 2 / 2 Risks - Exploitation likelihood is *Unlikely* with *Low* impact.

When a technical asset does not process any data assets, this is an indicator for an unnecessary technical asset (or for an incomplete model). This is also the case if the asset has no communication links (either outgoing or incoming).

Questions: 0 / 0 Questions

This chapter lists custom questions that arose during the threat modeling process.

No custom questions arose during the threat modeling process.

Identified Risks by Vulnerability category

In total **67 potential risks** have been identified during the threat modeling process of which **0 are rated as critical**, **2 as high**, **21 as elevated**, **38 as medium**, and **6 as low**.

These risks are distributed across **26 vulnerability categories**. The following sub-chapters of this section describe each identified risk category.

Asset Using Known Default or Hardcoded Credentials: 1 / 1 Risk

Description (Information Disclosure): CWE 1392

Technical assets tagged as intentional-misconfiguration that store confidential data represent an immediately exploitable attack vector. Default credentials (e.g. minioadmin/minioadmin, admin/admin, postgres/postgres) are published in vendor documentation and are the first credentials an attacker tries after reaching the service network.

Impact

Any attacker with network access can authenticate using default credentials, exfiltrate all stored data, modify records, or pivot into adjacent systems.

Detection Logic

In-scope technical assets tagged with intentional-misconfiguration that store data assets rated at confidential or above.

Risk Rating

High severity â€” default credentials plus confidential stored data equals a trivial full data breach with no exploitation skill required.

False Positives

Development or ephemeral test environments where no real PII or confidential data is present. Set the asset out_of_scope or remove the intentional-misconfiguration tag once credentials are rotated.

Mitigation (Operations): Rotate all default credentials before deployment; store secrets in a secrets manager

Replace all default credentials with randomly generated secrets (minimum 32 chars). Store in a secrets manager (Vault, AWS Secrets Manager, etc.) rather than environment variables or compose files. Add credential rotation to the pre-deployment checklist.

ASVS Chapter: V2 - Authentication Verification Requirements
Cheat Sheet: Authentication_Cheat_Sheet

Check

Are all service credentials rotated from vendor defaults? Are secrets stored in a dedicated secrets

manager rather than plain environment variables?

Risk Findings

The risk **Asset Using Known Default or Hardcoded Credentials** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.
Risk finding paragraphs are clickable and link to the corresponding chapter.

High Risk Severity

Exposed Default Credentials: **MinIO Object Storage** is tagged as intentional-misconfiguration and stores confidential data: Exploitation likelihood is *Very Likely* with *High* impact.

exposed-default-credentials@minio-storage

Unchecked

SQL/NoSQL-Injection: 2 / 2 Risks

Description (Tampering): CWE 89

When a database is accessed via database access protocols SQL/NoSQL-Injection risks might arise. The risk rating depends on the sensitivity technical asset itself and of the data assets processed.

Impact

If this risk is unmitigated, attackers might be able to modify SQL/NoSQL queries to steal and modify data and eventually further escalate towards a deeper system penetration via code executions.

Detection Logic

Database accessed via typical database access protocols by in-scope clients.

Risk Rating

The risk rating depends on the sensitivity of the data stored inside the database.

False Positives

Database accesses by queries not consisting of parts controllable by the caller can be considered as false positives after individual review.

Mitigation (Development): SQL/NoSQL-Injection Prevention

Try to use parameter binding to be safe from injection vulnerabilities. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

ASVS Chapter: V5 - Validation, Sanitization and Encoding Verification Requirements
Cheat Sheet: SQL_Injection_Prevention_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **SQL/NoSQL-Injection** was found **2 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

High Risk Severity

SQL/NoSQL-Injection risk at **API Server** against database **PostgreSQL Database** via **PostgreSQL Connection**: Exploitation likelihood is *Very Likely* with *High* impact.

sql-nosql-injection@api-server@postgresql-db@api-server>postgresql-connection

Unchecked

Elevated Risk Severity

SQL/NoSQL-Injection risk at **API Server** against database **Redis Cache** via **Redis Session Store**: Exploitation likelihood is *Very Likely* with *Medium* impact.

sql-nosql-injection@api-server@redis-cache@api-server>redis-session-store

Unchecked

Lateral Movement via Shared Runtime: 1 / 1 Risk

Description (Lateral Movement): n/a

Technical assets from different trust zones co-located on the same shared runtime (e.g. container platform, VM host, shared Kubernetes namespace) create a lateral movement path: a compromised low-trust asset on the runtime can escalate to high-trust neighbours without traversing a network boundary.

Impact

An attacker who compromises any asset on the shared runtime may pivot to other co-located assets, including those in higher trust zones.

Detection Logic

Shared runtimes that host technical assets belonging to at least two distinct trust boundaries are flagged.

Risk Rating

Severity depends on the sensitivity gap between the least and most sensitive co-located assets.

False Positives

Mitigation (Architecture): Runtime Isolation

Separate assets from different trust zones onto dedicated runtimes (separate node pools, dedicated VMs, distinct container namespaces with NetworkPolicy). Apply seccomp/AppArmor profiles and disable privilege escalation.

ASVS Chapter: V14 - Configuration Verification Requirements
Cheat Sheet: Docker_Security_Cheat_Sheet

Check

Are high-trust and low-trust assets co-located on the same shared runtime?

Risk Findings

The risk **Lateral Movement via Shared Runtime** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Lateral Movement via Shared Runtime on **Docker Host**: assets from 3 trust zones co-located: Exploitation likelihood is *Likely* with *High* impact.

lateral-movement-shared-runtime@docker-host

Unchecked

Missing Authentication: 2 / 2 Risks

Description (Elevation of Privilege): CWE 306

Technical assets (especially multi-tenant systems) should authenticate incoming requests when the asset processes sensitive data.

Impact

If this risk is unmitigated, attackers might be able to access or modify sensitive data in an unauthenticated way.

Detection Logic

In-scope technical assets (except load-balancer, reverse-proxy, service-registry, waf, ids, and ips and in-process calls) should authenticate incoming requests when the asset processes sensitive data. This is especially the case for all multi-tenant assets (there even non-sensitive ones).

Risk Rating

The risk rating (medium or high) depends on the sensitivity of the data sent across the communication link. Monitoring callers are exempted from this risk.

False Positives

Technical assets which do not process requests regarding functionality or data linked to end-users (customers) can be considered as false positives after individual review.

Mitigation (Architecture): Authentication of Incoming Requests

Apply an authentication method to the technical asset. To protect highly sensitive data consider the use of two-factor authentication for human users.

ASVS Chapter: V2 - Authentication Verification Requirements
Cheat Sheet: Authentication_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Missing Authentication** was found **2 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Missing Authentication covering communication link **HTTP to API Server** from **Nginx Reverse Proxy** to **API Server**: Exploitation likelihood is *Likely* with *High* impact.

missing-authentication@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Unchecked

Missing Authentication covering communication link **Route to ECS API** from **AWS API Gateway** to **API Server**: Exploitation likelihood is *Likely* with *High* impact.

missing-authentication@aws-api-gateway>route-to-ecs-api@aws-api-gateway@api-server

Unchecked

Missing Cloud Hardening: 3 / 3 Risks

Description (Tampering): CWE 1008

Cloud components should be hardened according to the cloud vendor best practices. This affects their configuration, auditing, and further areas.

Impact

If this risk is unmitigated, attackers might access cloud components in an unintended way.

Detection Logic

In-scope cloud components (either residing in cloud trust boundaries or more specifically tagged with cloud provider types).

Risk Rating

The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed.

False Positives

Cloud components not running parts of the target architecture can be considered as false positives after individual review.

Mitigation (Operations): Cloud Hardening

Apply hardening of all cloud components and services, taking special care to follow the individual risk descriptions (which depend on the cloud provider tags in the model).

For **Amazon Web Services (AWS)**: Follow the *CIS Benchmark for Amazon Web Services* (see also the automated checks of cloud audit tools like *"PacBot", "CloudSploit", "CloudMapper", "ScoutSuite", or "Prowler AWS CIS Benchmark Tool"*).
For EC2 and other servers running Amazon Linux, follow the *CIS Benchmark for Amazon Linux* and switch to IMDSv2.
For S3 buckets follow the *Security Best Practices for Amazon S3* at https://docs.aws.amazon.com/AmazonS3/latest/dev/security-best-practices.html to avoid accidental leakage.
Also take a look at some of these tools: https://github.com/toniblyx/my-arsenal-of-aws-security-tools

For **Microsoft Azure**: Follow the *CIS Benchmark for Microsoft Azure* (see also the automated checks of cloud audit tools like *"CloudSploit" or "ScoutSuite"*).

For **Google Cloud Platform**: Follow the *CIS Benchmark for Google Cloud Computing Platform* (see also the automated checks of cloud audit tools like *"CloudSploit" or "ScoutSuite"*).

For **Oracle Cloud Platform**: Follow the hardening best practices (see also the automated checks of cloud audit tools like *"CloudSploit"*).

ASVS Chapter: V1 - Architecture, Design and Threat Modeling Requirements
Cheat Sheet: Attack_Surface_Analysis_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Missing Cloud Hardening** was found **3 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Missing Cloud Hardening (AWS) risk at **AWS Cloud**: CIS Benchmark for AWS: Exploitation likelihood is *Unlikely* with *Very High* impact.

missing-cloud-hardening@aws-cloud@aws

Unchecked

Missing Cloud Hardening risk at **Docker Host**: Exploitation likelihood is *Unlikely* with *Very High* impact.

missing-cloud-hardening@docker-host

Unchecked

Medium Risk Severity

Missing Cloud Hardening risk at **External CI/CD**: Exploitation likelihood is *Unlikely* with *High* impact.

missing-cloud-hardening@external-cicd

Unchecked

Missing Content-Security-Policy Header on Web Entry Point: 1 / 1 Risk

Description (Information Disclosure): CWE 693

Web entry points (reverse proxies and web applications) that serve browser-based clients and process authentication tokens without a Content-Security-Policy (CSP) header are vulnerable to XSS-facilitated token theft. An attacker who achieves XSS in the browser can read localStorage/sessionStorage tokens and replay them without any client-side protection if CSP is absent.

Impact

An attacker who successfully injects client-side script can exfiltrate authentication tokens, impersonate users, and access all data the victim can access. Without CSP, there is no browser-enforced barrier against inline script execution or data exfiltration.

Detection Logic

In-scope technical assets with reverse-proxy or web-application technology that process PII data assets and are not tagged with has-csp. The has-csp tag serves as the in-model confirmation that CSP is deployed and verified.

Risk Rating

Medium baseline. Elevated when the application uses long-lived tokens (>1 h TTL) stored in browser-accessible storage, or when no WAF is present to block reflected XSS.

False Positives

Add the tag has-csp to the technical asset once a Content-Security-Policy header is verified in production responses. Also false positive when the application exclusively uses HttpOnly cookies for session management and no JS-accessible token storage is used.

Mitigation (Operations): Configure a Content-Security-Policy response header on the web entry point

Add a strict Content-Security-Policy header to all responses served to browser clients. At minimum use: default-src 'self'; script-src 'self'; object-src 'none'; base-uri 'none'. Combine with Subresource Integrity (SRI) for external scripts and avoid inline event handlers. Once deployed, tag the asset has-csp to suppress this finding.

ASVS Chapter: V14 - Configuration Verification Requirements
Cheat Sheet: Content_Security_Policy_Cheat_Sheet

Check

Is a Content-Security-Policy response header present and non-trivial on all pages served to browsers? Use browser DevTools or securityheaders.com to verify.

Risk Findings

The risk **Missing Content-Security-Policy Header on Web Entry Point** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Missing Content-Security-Policy: **Nginx Reverse Proxy** serves browser clients handling auth tokens without a verified CSP header: Exploitation likelihood is *Likely* with *Medium* impact.

missing-csp-header@nginx-proxy

Unchecked

Missing Hardening: 3 / 3 Risks

Description (Tampering): CWE 16

Technical assets with a Relative Attacker Attractiveness (RAA) value of 55 % or higher should be explicitly hardened taking best practices and vendor hardening guides into account.

Impact

If this risk remains unmitigated, attackers might be able to easier attack high-value targets.

Detection Logic

In-scope technical assets with RAA values of 55 % or higher. Generally for high-value targets like data stores, application servers, identity providers and ERP systems this limit is reduced to 40 %

Risk Rating

The risk rating depends on the sensitivity of the data processed in the technical asset.

False Positives

Usually no false positives.

Mitigation (Operations): System Hardening

Try to apply all hardening best practices (like CIS benchmarks, OWASP recommendations, vendor recommendations, DevSec Hardening Framework, DBSAT for Oracle databases, and others).

ASVS Chapter: V14 - Configuration Verification Requirements
Cheat Sheet: Attack_Surface_Analysis_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Missing Hardening** was found **3 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Missing Hardening risk at **AWS RDS PostgreSQL**: Exploitation likelihood is *Likely* with *Medium* impact.

missing-hardening@rds-postgresql

Unchecked

Missing Hardening risk at **ECS Container Platform**: Exploitation likelihood is *Likely* with *Medium* impact.

missing-hardening@ecs-platform

Unchecked

Missing Hardening risk at **PostgreSQL Database**: Exploitation likelihood is *Likely* with *Medium* impact.

missing-hardening@postgresql-db

Unchecked

Path-Traversal: 3 / 3 Risks

Description (Information Disclosure): CWE 22

When a filesystem is accessed Path-Traversal or Local-File-Inclusion (LFI) risks might arise. The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed.

Impact

If this risk is unmitigated, attackers might be able to read sensitive files (configuration data, key/credential files, deployment files, business data files, etc.) from the filesystem of affected components.

Detection Logic

Filesystems accessed by in-scope callers.

Risk Rating

The risk rating depends on the sensitivity of the data stored inside the technical asset.

False Positives

File accesses by filenames not consisting of parts controllable by the caller can be considered as false positives after individual review.

Mitigation (Development): Path-Traversal Prevention

Before accessing the file cross-check that it resides in the expected folder and is of the expected type and filename/suffix. Try to use a mapping if possible instead of directly accessing by a filename which is (partly or fully) provided by the caller. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

ASVS Chapter: V12 - File and Resources Verification Requirements
Cheat Sheet: Input_Validation_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Path-Traversal** was found **3 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Path-Traversal risk at **API Server** against filesystem **MinIO Object Storage** via **MinIO File Storage**: Exploitation likelihood is *Very Likely* with *Medium* impact.

path-traversal@api-server@minio-storage@api-server>minio-file-storage

Unchecked

Path-Traversal risk at **ECS Container Platform** against filesystem **AWS ECR Container Registry** via **Pull Images from ECR**: Exploitation likelihood is *Likely* with *Medium* impact.

path-traversal@ecs-platform@container-registry@ecs-platform>pull-images-from-ecr

Unchecked

Path-Traversal risk at **GitHub Actions Build Pipeline** against filesystem **AWS ECR Container Registry** via **Push Image to Registry**: Exploitation likelihood is *Likely* with *Medium* impact.

path-traversal@build-pipeline@container-registry@build-pipeline>push-image-to-registry

Unchecked

Server-Side Request Forgery (SSRF): 8 / 8 Risks

Description (Information Disclosure): CWE 918

When a server system (i.e. not a client) is accessing other server systems via typical web protocols Server-Side Request Forgery (SSRF) or Local-File-Inclusion (LFI) or Remote-File-Inclusion (RFI) risks might arise.

Impact

If this risk is unmitigated, attackers might be able to access sensitive services or files of network-reachable components by modifying outgoing calls of affected components.

Detection Logic

In-scope non-client systems accessing (using outgoing communication links) targets with either HTTP or HTTPS protocol.

Risk Rating

The risk rating (low or medium) depends on the sensitivity of the data assets receivable via web protocols from targets within the same network trust-boundary as well on the sensitivity of the data assets receivable via web protocols from the target asset itself. Also for cloud-based environments the exploitation impact is at least medium, as cloud backend services can be attacked via SSRF.

False Positives

Servers not sending outgoing web requests can be considered as false positives after review.

Mitigation (Development): SSRF Prevention

Try to avoid constructing the outgoing target URL with caller controllable values. Alternatively use a mapping (whitelist) when accessing outgoing URLs instead of creating them including caller controllable values. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

ASVS Chapter: V12 - File and Resources Verification Requirements
Cheat Sheet: Server_Side_Request_Forgery_Prevention_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Server-Side Request Forgery (SSRF)** was found **8 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Server-Side Request Forgery (SSRF) risk at **API Server** server-side web-requesting the target **MinIO Object Storage** via **MinIO File Storage**: Exploitation likelihood is *Likely* with *Medium* impact.

server-side-request-forgery@api-server@minio-storage@api-server>minio-file-storage

Unchecked

Server-Side Request Forgery (SSRF) risk at **Lambda Email Notifier** server-side web-requesting the target **AWS SES** via **SES Email Send**: Exploitation likelihood is *Likely* with *Medium* impact.

server-side-request-forgery@lambda-notifier@aws-ses@lambda-notifier>ses-email-send

Unchecked

Medium Risk Severity

Server-Side Request Forgery (SSRF) risk at **ECS Container Platform** server-side web-requesting the target **AWS ECR Container Registry** via **Pull Images from ECR**: Exploitation likelihood is *Unlikely* with *Medium* impact.

server-side-request-forgery@ecs-platform@container-registry@ecs-platform>pull-images-from-ecr

Unchecked

Server-Side Request Forgery (SSRF) risk at **GitHub Actions Build Pipeline** server-side web-requesting the target **AWS ECR Container Registry** via **Push Image to Registry**: Exploitation likelihood is *Unlikely* with *Medium* impact.

server-side-request-forgery@build-pipeline@container-registry@build-pipeline>push-image-to-registry

Unchecked

Server-Side Request Forgery (SSRF) risk at **VaultNote Source Repository** server-side web-requesting the target **GitHub Actions Build Pipeline** via **Push to Pipeline**: Exploitation likelihood is *Unlikely* with *Medium* impact.

server-side-request-forgery@source-repo@build-pipeline@source-repo>push-to-pipeline

Unchecked

Server-Side Request Forgery (SSRF) risk at **LLM Note Summarizer** server-side web-requesting the target **Note RAG Pipeline** via **Call RAG Pipeline**: Exploitation likelihood is *Likely* with *Low* impact.

server-side-request-forgery@llm-summarizer@rag-pipeline@llm-summarizer>call-rag-pipeline

Unchecked

Server-Side Request Forgery (SSRF) risk at **LLM Note Summarizer** server-side web-requesting the target **VaultNote Vector Store** via **Query Vector Store**: Exploitation likelihood is *Likely* with *Low* impact.

server-side-request-forgery@llm-summarizer@vector-store@llm-summarizer>query-vector-store

Unchecked

Server-Side Request Forgery (SSRF) risk at **Note RAG Pipeline** server-side web-requesting the target **VaultNote Vector Store** via **Retrieve from Vector Store**: Exploitation likelihood is *Likely* with *Low* impact.

server-side-request-forgery@rag-pipeline@vector-store@rag-pipeline>retrieve-from-vector-store

Unchecked

Unencrypted Communication: 4 / 4 Risks

Description (Information Disclosure): CWE 319

Due to the confidentiality and/or integrity rating of the data assets transferred over the communication link this connection must be encrypted.

Impact

If this risk is unmitigated, network attackers might be able to to eavesdrop on unencrypted sensitive data sent between components.

Detection Logic

Unencrypted technical communication links of in-scope technical assets (excluding monitoring traffic as well as local-file-access, in-process-library-call and inter-process-communication) transferring sensitive data.

Risk Rating

Depending on the confidentiality rating of the transferred data-assets either medium or high risk.

False Positives

When all sensitive data sent over the communication link is already fully encrypted on document or data level. Also intra-container/pod communication can be considered false positive when container orchestration platform handles encryption.

Mitigation (Operations): Encryption of Communication Links

Apply transport layer encryption to the communication link.

ASVS Chapter: V9 - Communication Verification Requirements
Cheat Sheet: Transport_Layer_Protection_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Unencrypted Communication** was found **4 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Unencrypted Communication named **MinIO File Storage** between **API Server** and **MinIO Object Storage** transferring authentication data (like credentials, token, session-id, etc.): Exploitation likelihood is *Likely* with *High* impact.

unencrypted-communication@api-server>minio-file-storage@api-server@minio-storage

Unchecked

Unencrypted Communication named **PostgreSQL Connection** between **API Server** and **PostgreSQL Database** transferring authentication data (like credentials, token, session-id, etc.): Exploitation likelihood is *Likely* with *High* impact.

unencrypted-communication@api-server>postgresql-connection@api-server@postgresql-db

Unchecked

Unencrypted Communication named **Redis Session Store** between **API Server** and **Redis Cache** transferring authentication data (like credentials, token, session-id, etc.): Exploitation likelihood is *Likely* with *High* impact.

unencrypted-communication@api-server>redis-session-store@api-server@redis-cache

Unchecked

Unencrypted Communication named **HTTP to API Server** between **Nginx Reverse Proxy** and **API Server**: Exploitation likelihood is *Likely* with *Medium* impact.

unencrypted-communication@nginx-proxy>http-to-api-server@nginx-proxy@api-server

In Progress 2026-04-18 Security Team VAULT-102

Migrating Nginx-to-API communication to mTLS. A separate CA will be provisioned inside the Docker network. PR #47 is in review; target completion sprint 22.

Unguarded Access From Internet: 5 / 5 Risks

Description (Elevation of Privilege): CWE 501

Internet-exposed assets must be guarded by a protecting service, application, or reverse-proxy.

Impact

If this risk is unmitigated, attackers might be able to directly attack sensitive systems without any hardening components in-between due to them being directly exposed on the internet.

Detection Logic

In-scope technical assets (excluding load-balancer) with confidentiality rating of confidential (or higher) or with integrity rating of critical (or higher) when accessed directly from the internet. All web-server, web-application, reverse-proxy, waf, and gateway assets are exempted from this risk when they do not consist of custom developed code and the data-flow only consists of HTTP or FTP protocols. Access from monitoring systems as well as VPN-protected connections are exempted.

Risk Rating

The matching technical assets are at low risk. When either the confidentiality rating is strictly-confidential or the integrity rating is mission-critical, the risk-rating is considered medium. For assets with RAA values higher than 40 % the risk-rating increases.

False Positives

When other means of filtering client requests are applied equivalent of reverse-proxy, waf, or gateway components.

Mitigation (Architecture): Encapsulation of Technical Asset

Encapsulate the asset behind a guarding service, application, or reverse-proxy. For admin maintenance a bastion-host should be used as a jump-server. For file transfer a store-and-forward-host should be used as an indirect file exchange platform.

ASVS Chapter: V1 - Architecture, Design and Threat Modeling Requirements
Cheat Sheet: Attack_Surface_Analysis_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Unguarded Access From Internet** was found **5 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.
Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Unguarded Access from Internet of **API Server** by **AWS API Gateway** via **Route to ECS API**: Exploitation likelihood is *Very Likely* with *Medium* impact.
unguarded-access-from-internet@api-server@aws-api-gateway@aws-api-gateway>route-to-ecs-api
Unchecked

Medium Risk Severity

Unguarded Access from Internet of **AWS ECR Container Registry** by **GitHub Actions Build Pipeline** via **Push Image to Registry**: Exploitation likelihood is *Very Likely* with *Low* impact.
unguarded-access-from-internet@container-registry@build-pipeline@build-pipeline>push-image-to-registry
Unchecked

Unguarded Access from Internet of **GitHub Actions Build Pipeline** by **VaultNote Source Repository** via **Push to Pipeline**: Exploitation likelihood is *Very Likely* with *Low* impact.
unguarded-access-from-internet@build-pipeline@source-repo@source-repo>push-to-pipeline
Unchecked

Unguarded Access from Internet of **Note RAG Pipeline** by **LLM Note Summarizer** via **Call RAG Pipeline**: Exploitation likelihood is *Very Likely* with *Low* impact.
unguarded-access-from-internet@rag-pipeline@llm-summarizer@llm-summarizer>call-rag-pipeline
Unchecked

Unguarded Access from Internet of **VaultNote Vector Store** by **LLM Note Summarizer** via **Query Vector Store**: Exploitation likelihood is *Very Likely* with *Low* impact.
unguarded-access-from-internet@vector-store@llm-summarizer@llm-summarizer>query-vector-store
Unchecked

Unguarded Direct Datastore Access: 3 / 3 Risks

Description (Elevation of Privilege): CWE 501

Data stores accessed across trust boundaries must be guarded by some protecting service or application.

Impact

If this risk is unmitigated, attackers might be able to directly attack sensitive data stores without any protecting components in-between.

Detection Logic

In-scope technical assets of type datastore (except identity-store-ldap when accessed from identity-provider and file-server when accessed via file transfer protocols) with confidentiality rating of confidential (or higher) or with integrity rating of critical (or higher) which have incoming data-flows from assets outside across a network trust-boundary. DevOps config and deployment access is excluded from this risk.

Risk Rating

The matching technical assets are at low risk. When either the confidentiality rating is strictly-confidential or the integrity rating is mission-critical, the risk-rating is considered medium. For assets with RAA values higher than 40 % the risk-rating increases.

False Positives

When the caller is considered fully trusted as if it was part of the datastore itself.

Mitigation (Architecture): Encapsulation of Datastore

Encapsulate the datastore access behind a guarding service or application.

ASVS Chapter: V1 - Architecture, Design and Threat Modeling Requirements
Cheat Sheet: Attack_Surface_Analysis_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Unguarded Direct Datastore Access** was found **3 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Unguarded Direct Datastore Access of **PostgreSQL Database** by **API Server** via **PostgreSQL Connection**: Exploitation likelihood is *Likely* with *Medium* impact.

unguarded-direct-datastore-access@api-server>postgresql-connection@api-server@postgresql-db

Unchecked

Medium Risk Severity

Unguarded Direct Datastore Access of **MinIO Object Storage** by **API Server** via **MinIO File Storage**: Exploitation likelihood is *Likely* with *Low* impact.

unguarded-direct-datastore-access@api-server>minio-file-storage@api-server@minio-storage

Unchecked

Unguarded Direct Datastore Access of **Redis Cache** by **API Server** via **Redis Session Store**: Exploitation likelihood is *Likely* with *Low* impact.

unguarded-direct-datastore-access@api-server>redis-session-store@api-server@redis-cache

Unchecked

Accidental Secret Leak: 1 / 1 Risk

Description (Information Disclosure): CWE 200

Sourcecode repositories (including their histories) as well as artifact registries can accidentally contain secrets like checked-in or packaged-in passwords, API tokens, certificates, crypto keys, etc.

Impact

If this risk is unmitigated, attackers which have access to affected sourcecode repositories or artifact registries might find secrets accidentally checked-in.

Detection Logic

In-scope sourcecode repositories and artifact registries.

Risk Rating

The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed.

False Positives

Usually no false positives.

Mitigation (Operations): Build Pipeline Hardening

Establish measures preventing accidental check-in or package-in of secrets into sourcecode repositories and artifact registries. This starts by using good .gitignore and .dockerignore files, but does not stop there. See for example tools like \"*git-secrets*\" or \"*Talisman*\" to have check-in preventive measures for secrets. Consider also to regularly scan your repositories for secrets accidentally checked-in using scanning tools like *"gitleaks"* or *"gitrob"*.

ASVS Chapter: V14 - Configuration Verification Requirements
Cheat Sheet: Attack_Surface_Analysis_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Accidental Secret Leak** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Accidental Secret Leak(Git) risk at **VaultNote Source Repository**: Git Leak Prevention: Exploitation likelihood is *Unlikely* with *Medium* impact.

accidental-secret-leak@source-repo

Unchecked

Code Backdooring: 3 / 3 Risks

Description (Tampering): CWE 912

For each build pipeline component Code Backdooring risks might arise where attackers compromise the build pipeline in order to let backdoored artifacts be shipped into production. Aside from direct code backdooring this includes backdooring of dependencies and even of more lower-level build infrastructure, like backdooring compilers (similar to what the XcodeGhost malware did) or dependencies.

Impact

If this risk remains unmitigated, attackers might be able to execute code on and completely takeover production environments.

Detection Logic

In-scope development relevant technical assets which are either accessed by out-of-scope unmanaged developer clients and/or are directly accessed by any kind of internet-located (non-VPN) component or are themselves directly located on the internet.

Risk Rating

The risk rating depends on the confidentiality and integrity rating of the code being handled and deployed as well as the placement/calling of this technical asset on/from the internet.

False Positives

When the build-pipeline and sourcecode-repo is not exposed to the internet and considered fully trusted (which implies that all accessing clients are also considered fully trusted in terms of their patch management and applied hardening, which must be equivalent to a managed developer client environment) this can be considered a false positive after individual review.

Mitigation (Operations): Build Pipeline Hardening

Reduce the attack surface of backdooring the build pipeline by not directly exposing the build pipeline components on the public internet and also not exposing it in front of unmanaged (out-of-scope) developer clients.Also consider the use of code signing to prevent code modifications.

ASVS Chapter: V10 - Malicious Code Verification Requirements
Cheat Sheet: Vulnerable_Dependency_Management_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Code Backdooring** was found **3 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Code Backdooring risk at **AWS ECR Container Registry**: Exploitation likelihood is *Unlikely* with *High* impact.

code-backdooring@container-registry

Unchecked

Code Backdooring risk at **GitHub Actions Build Pipeline**: Exploitation likelihood is *Unlikely* with *High* impact.

code-backdooring@build-pipeline

Unchecked

Code Backdooring risk at **VaultNote Source Repository**: Exploitation likelihood is *Unlikely* with *High* impact.

code-backdooring@source-repo

Unchecked

Container Base Image Backdooring: 7 / 7 Risks

Description (Tampering): CWE 912

When a technical asset is built using container technologies, Base Image Backdooring risks might arise where base images and other layers used contain vulnerable components or backdoors.

See for example:
https://techcrunch.com/2018/06/15/tainted-crypto-mining-containers-pulled-from-docker-hub/

Impact

If this risk is unmitigated, attackers might be able to deeply persist in the target system by executing code in deployed containers.

Detection Logic

In-scope technical assets running as containers.

Risk Rating

The risk rating depends on the sensitivity of the technical asset itself and of the data assets.

False Positives

Fully trusted (i.e. reviewed and cryptographically signed or similar) base images of containers can be considered as false positives after individual review.

Mitigation (Operations): Container Infrastructure Hardening

Apply hardening of all container infrastructures (see for example the *CIS-Benchmarks for Docker and Kubernetes* and the *Docker Bench for Security*). Use only trusted base images of the original vendors, verify digital signatures and apply image creation best practices. Also consider using Google's *Distroless* base images or otherwise very small base images. Regularly execute container image scans with tools checking the layers for vulnerable components.

ASVS Chapter: V10 - Malicious Code Verification Requirements
Cheat Sheet: Docker_Security_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS/CSVS applied?

Risk Findings

The risk **Container Base Image Backdooring** was found **7 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Container Base Image Backdooring risk at **API Server**: Exploitation likelihood is *Unlikely* with *High* impact.

container-baseimage-backdooring@api-server

Unchecked

Container Base Image Backdooring risk at **Nginx Reverse Proxy**: Exploitation likelihood is *Unlikely* with *High* impact.

container-baseimage-backdooring@nginx-proxy

Unchecked

Container Base Image Backdooring risk at **PostgreSQL Database**: Exploitation likelihood is *Unlikely* with *High* impact.

container-baseimage-backdooring@postgresql-db

Unchecked

Container Base Image Backdooring risk at **MinIO Object Storage**: Exploitation likelihood is *Unlikely* with *Medium* impact.

container-baseimage-backdooring@minio-storage

Unchecked

Container Base Image Backdooring risk at **Note RAG Pipeline**: Exploitation likelihood is *Unlikely* with *Medium* impact.

container-baseimage-backdooring@rag-pipeline

Unchecked

Container Base Image Backdooring risk at **Redis Cache**: Exploitation likelihood is *Unlikely* with *Medium* impact.

container-baseimage-backdooring@redis-cache

Unchecked

Container Base Image Backdooring risk at **VaultNote Vector Store**: Exploitation likelihood is *Unlikely* with *Medium* impact.

container-baseimage-backdooring@vector-store

Unchecked

Container Platform Escape: 1 / 1 Risk

Description (Elevation of Privilege): CWE 1008

Container platforms are especially interesting targets for attackers as they host big parts of a containerized runtime infrastructure. When not configured and operated with security best practices in mind, attackers might exploit a vulnerability inside an container and escape towards the platform as highly privileged users. These scenarios might give attackers capabilities to attack every other container as owning the container platform (via container escape attacks) equals to owning every container.

Impact

If this risk is unmitigated, attackers which have successfully compromised a container (via other vulnerabilities) might be able to deeply persist in the target system by executing code in many deployed containers and the container platform itself.

Detection Logic

In-scope container platforms.

Risk Rating

The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed.

False Positives

Container platforms not running parts of the target architecture can be considered as false positives after individual review.

Mitigation (Operations): Container Infrastructure Hardening

Apply hardening of all container infrastructures. See for example the *CIS-Benchmarks for Docker and Kubernetes* as well as the *Docker Bench for Security* (https://github.com/docker/docker-bench-security) or *InSpec Checks for Docker and Kubernetes* (https://github.com/dev-sec/cis-docker-benchmark and https://github.com/dev-sec/cis-kubernetes-benchmark). Use only trusted base images, verify digital signatures and apply image creation best practices. Also consider using Google's **Distroless base images or otherwise very small base images. Apply namespace isolation and nod affinity to separate pods from each other in terms of access and nodes the same style as you separate data.**

ASVS Chapter: V14 - Configuration Verification Requirements
Cheat Sheet: Docker_Security_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS or CSVS chapter applied?

Risk Findings

The risk **Container Platform Escape** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Container Platform Escape risk at **ECS Container Platform**: Exploitation likelihood is *Unlikely* with *High* impact.

container-platform-escape@ecs-platform

Unchecked

Missing Build Infrastructure: 1 / 1 Risk

Description (Tampering): CWE 1127

The modeled architecture does not contain a build infrastructure (devops-client, sourcecode-repo, build-pipeline, etc.), which might be the risk of a model missing critical assets (and thus not seeing their risks). If the architecture contains custom-developed parts, the pipeline where code gets developed and built needs to be part of the model.

Impact

If this risk is unmitigated, attackers might be able to exploit risks unseen in this threat model due to critical build infrastructure components missing in the model.

Detection Logic

Models with in-scope custom-developed parts missing in-scope development (code creation) and build infrastructure components (devops-client, sourcecode-repo, build-pipeline, etc.).

Risk Rating

The risk rating depends on the highest sensitivity of the in-scope assets running custom-developed parts.

False Positives

Models not having any custom-developed parts can be considered as false positives after individual review.

Mitigation (Architecture): Build Pipeline Hardening

Include the build infrastructure in the model.

ASVS Chapter: V1 - Architecture, Design and Threat Modeling Requirements
Cheat Sheet: Attack_Surface_Analysis_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Missing Build Infrastructure** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.
Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Missing Build Infrastructure in the threat model (referencing asset **API Server** as an example): Exploitation likelihood is *Unlikely* with *Medium* impact.

missing-build-infrastructure@api-server

Unchecked

Missing Identity Store: 1 / 1 Risk

Description (Spoofing): CWE 287

The modeled architecture does not contain an identity store, which might be the risk of a model missing critical assets (and thus not seeing their risks).

Impact

If this risk is unmitigated, attackers might be able to exploit risks unseen in this threat model in the identity provider/store that is currently missing in the model.

Detection Logic

Models with authenticated data-flows authorized via end user identity missing an in-scope identity store.

Risk Rating

The risk rating depends on the sensitivity of the end user-identity authorized technical assets and their data assets processed.

False Positives

Models only offering data/services without any real authentication need can be considered as false positives after individual review.

Mitigation (Architecture): Identity Store

Include an identity store in the model if the application has a login.

ASVS Chapter: V2 - Authentication Verification Requirements
Cheat Sheet: Authentication_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Missing Identity Store** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Missing Identity Store in the threat model (referencing asset **Nginx Reverse Proxy** as an example): Exploitation likelihood is *Unlikely* with *Medium* impact.

missing-identity-store@nginx-proxy

Unchecked

Missing Two-Factor Authentication (2FA): 1 / 1 Risk

Description (Elevation of Privilege): CWE 308

Technical assets (especially multi-tenant systems) should authenticate incoming requests with two-factor (2FA) authentication when the asset processes or stores highly sensitive data (in terms of confidentiality, integrity, and availability) and is accessed by humans.

Impact

If this risk is unmitigated, attackers might be able to access or modify highly sensitive data without strong authentication.

Detection Logic

In-scope technical assets (except load-balancer, reverse-proxy, waf, ids, and ips) should authenticate incoming requests via two-factor authentication (2FA) when the asset processes or stores highly sensitive data (in terms of confidentiality, integrity, and availability) and is accessed by a client used by a human user.

Risk Rating

medium

False Positives

Technical assets which do not process requests regarding functionality or data linked to end-users (customers) can be considered as false positives after individual review.

Mitigation (Business Side): Authentication with Second Factor (2FA)

Apply an authentication method to the technical asset protecting highly sensitive data via two-factor authentication for human users.

ASVS Chapter: V2 - Authentication Verification Requirements
Cheat Sheet: Multifactor_Authentication_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Missing Two-Factor Authentication (2FA)** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Missing Two-Factor Authentication covering communication link **HTTP to API Server** from **Browser SPA** forwarded via **Nginx Reverse Proxy** to **API Server**: Exploitation likelihood is *Unlikely* with *Medium* impact.

missing-authentication-second-factor@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Unchecked

Missing Vault (Secret Storage): 1 / 1 Risk

Description (Information Disclosure): CWE 522

In order to avoid the risk of secret leakage via config files (when attacked through vulnerabilities being able to read files like Path-Traversal and others), it is best practice to use a separate hardened process with proper authentication, authorization, and audit logging to access config secrets (like credentials, private keys, client certificates, etc.). This component is usually some kind of Vault.

Impact

If this risk is unmitigated, attackers might be able to easier steal config secrets (like credentials, private keys, client certificates, etc.) once a vulnerability to access files is present and exploited.

Detection Logic

Models without a Vault (Secret Storage).

Risk Rating

The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed.

False Positives

Models where no technical assets have any kind of sensitive config data to protect can be considered as false positives after individual review.

Mitigation (Architecture): Vault (Secret Storage)

Consider using a Vault (Secret Storage) to securely store and access config secrets (like credentials, private keys, client certificates, etc.).

ASVS Chapter: V6 - Stored Cryptography Verification Requirements
Cheat Sheet: Cryptographic_Storage_Cheat_Sheet

Check

GetAttribute a Vault (Secret Storage) in place?

Risk Findings

The risk **Missing Vault (Secret Storage)** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Missing Vault (Secret Storage) in the threat model (referencing asset **API Server** as an example): Exploitation likelihood is *Unlikely* with *Medium* impact.

missing-vault@api-server

Unchecked

Missing Web Application Firewall (WAF): 1 / 1 Risk

Description (Tampering): CWE 1008

To have a first line of filtering defense, security architectures with web-services or web-applications should include a WAF in front of them. Even though a WAF is not a replacement for security (all components must be secure even without a WAF) it adds another layer of defense to the overall system by delaying some attacks and having easier attack alerting through it.

Impact

If this risk is unmitigated, attackers might be able to apply standard attack pattern tests at great speed without any filtering.

Detection Logic

In-scope web-services and/or web-applications accessed across a network trust boundary not having a Web Application Firewall (WAF) in front of them.

Risk Rating

The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed.

False Positives

Targets only accessible via WAFs or reverse proxies containing a WAF component (like ModSecurity) can be considered as false positives after individual review.

Mitigation (Operations): Web Application Firewall (WAF)

Consider placing a Web Application Firewall (WAF) in front of the web-services and/or web-applications. For cloud environments many cloud providers offer pre-configured WAFs. Even reverse proxies can be enhances by a WAF component via ModSecurity plugins.

ASVS Chapter: V1 - Architecture, Design and Threat Modeling Requirements
Cheat Sheet: Virtual_Patching_Cheat_Sheet

Check

GetAttribute a Web Application Firewall (WAF) in place?

Risk Findings

The risk **Missing Web Application Firewall (WAF)** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.
Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Missing Web Application Firewall (WAF) risk at **API Server**: Exploitation likelihood is *Unlikely* with *Medium* impact.

missing-waf@api-server

Unchecked

Mixed Targets on Shared Runtime: 1 / 1 Risk

Description (Elevation of Privilege): CWE 1008

Different attacker targets (like frontend and backend/datastore components) should not be running on the same shared (underlying) runtime.

Impact

If this risk is unmitigated, attackers successfully attacking other components of the system might have an easy path towards more valuable targets, as they are running on the same shared runtime.

Detection Logic

Shared runtime running technical assets of different trust-boundaries is at risk. Also mixing backend/datastore with frontend components on the same shared runtime is considered a risk.

Risk Rating

The risk rating (low or medium) depends on the confidentiality, integrity, and availability rating of the technical asset running on the shared runtime.

False Positives

When all assets running on the shared runtime are hardened and protected to the same extend as if all were containing/processing highly sensitive data.

Mitigation (Operations): Runtime Separation

Use separate runtime environments for running different target components or apply similar separation styles to prevent load- or breach-related problems originating from one more attacker-facing asset impacts also the other more critical rated backend/datastore assets.

ASVS Chapter: V1 - Architecture, Design and Threat Modeling Requirements
Cheat Sheet: Attack_Surface_Analysis_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Mixed Targets on Shared Runtime** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Mixed Targets on Shared Runtime named **Docker Host** might enable attackers moving from one less valuable target to a more valuable one: Exploitation likelihood is *Unlikely* with *Medium* impact.

mixed-targets-on-shared-runtime@docker-host

Accepted 2026-04-10 Infrastructure Team VAULT-95

Docker containers run without privileged mode and with no host path mounts except the named volumes. The Docker socket is not mounted into any container. Risk is accepted for this demo environment.

Unchecked Deployment: 3 / 3 Risks

Description (Tampering): CWE 1127

For each build-pipeline component Unchecked Deployment risks might arise when the build-pipeline does not include established DevSecOps best-practices. DevSecOps best-practices scan as part of CI/CD pipelines for vulnerabilities in source- or byte-code, dependencies, container layers, and dynamically against running test systems. There are several open-source and commercial tools existing in the categories DAST, SAST, and IAST.

Impact

If this risk remains unmitigated, vulnerabilities in custom-developed software or their dependencies might not be identified during continuous deployment cycles.

Detection Logic

All development-relevant technical assets.

Risk Rating

The risk rating depends on the highest rating of the technical assets and data assets processed by deployment-receiving targets.

False Positives

When the build-pipeline does not build any software components it can be considered a false positive after individual review.

Mitigation (Architecture): Build Pipeline Hardening

Apply DevSecOps best-practices and use scanning tools to identify vulnerabilities in source- or byte-code,dependencies, container layers, and optionally also via dynamic scans against running test systems.

ASVS Chapter: V14 - Configuration Verification Requirements
Cheat Sheet: Vulnerable_Dependency_Management_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Unchecked Deployment** was found **3 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Unchecked Deployment risk at **GitHub Actions Build Pipeline**: Exploitation likelihood is *Unlikely* with *Medium* impact.

unchecked-deployment@build-pipeline

Unchecked

Unchecked Deployment risk at **VaultNote Source Repository**: Exploitation likelihood is *Unlikely* with *Medium* impact.

unchecked-deployment@source-repo

Unchecked

Low Risk Severity

Unchecked Deployment risk at **AWS ECR Container Registry**: Exploitation likelihood is *Unlikely* with *Low* impact.

unchecked-deployment@container-registry

Unchecked

Unencrypted Technical Assets: 5 / 5 Risks

Description (Information Disclosure): CWE 311

Due to the confidentiality rating of the technical asset itself and/or the stored data assets this technical asset must be encrypted. The risk rating depends on the sensitivity technical asset itself and of the data assets stored.

Impact

If this risk is unmitigated, attackers might be able to access unencrypted data when successfully compromising sensitive components.

Detection Logic

In-scope unencrypted technical assets (excluding reverse-proxy, load-balancer, waf, ids, ips and embedded components like library) storing data assets rated at least as confidential or critical. For technical assets storing data assets rated as strictly-confidential or mission-critical the encryption must be of type data-with-end-user-individual-key.

Risk Rating

Depending on the confidentiality rating of the stored data-assets either medium or high risk.

False Positives

When all sensitive data stored within the asset is already fully encrypted on document or data level.

Mitigation (Operations): Encryption of Technical Asset

Apply encryption to the technical asset.

ASVS Chapter: V6 - Stored Cryptography Verification Requirements
Cheat Sheet: Cryptographic_Storage_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Unencrypted Technical Assets** was found **5 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Unencrypted Technical Asset named **AWS RDS PostgreSQL** missing end user individual encryption with data-with-end-user-individual-key: Exploitation likelihood is *Unlikely* with *High* impact.

unencrypted-asset@rds-postgresql

Unchecked

Unencrypted Technical Asset named **AWS S3 Notes Bucket**: Exploitation likelihood is *Unlikely* with *Medium* impact.

unencrypted-asset@s3-notes-bucket

Unchecked

Unencrypted Technical Asset named **Redis Cache**: Exploitation likelihood is *Unlikely* with *Medium* impact.

unencrypted-asset@redis-cache

Unchecked

Unencrypted Technical Asset named **PostgreSQL Database** missing end user individual encryption with data-with-end-user-individual-key: Exploitation likelihood is *Unlikely* with *High* impact.

unencrypted-asset@postgresql-db

Accepted 2026-04-15 Infrastructure Team VAULT-89

The PostgreSQL volume runs on a host-level LUKS-encrypted partition in all production deployments. The Threagile finding is acknowledged as informational for this demo environment which uses unencrypted storage. Production database TDE via pgcrypto will be evaluated in Q3.

Unencrypted Technical Asset named **MinIO Object Storage**: Exploitation likelihood is *Unlikely* with *Medium* impact.

unencrypted-asset@minio-storage

In Progress 2026-04-18 Security Team VAULT-110

MinIO server-side encryption (SSE-S3) is being evaluated. Default minioadmin credentials will be rotated before production deployment. This finding is a CRITICAL blocker for launch. Tracked in VAULT-110.

DoS-risky Access Across Trust-Boundary: 3 / 3 Risks

Description (Denial of Service): CWE 400

Assets accessed across trust boundaries with critical or mission-critical availability rating are more prone to Denial-of-Service (DoS) risks.

Impact

If this risk remains unmitigated, attackers might be able to disturb the availability of important parts of the system.

Detection Logic

In-scope technical assets (excluding load-balancer) with availability rating of critical or higher which have incoming data-flows across a network trust-boundary (excluding devops usage).

Risk Rating

Matching technical assets with availability rating of critical or higher are at low risk. When the availability rating is mission-critical and neither a VPN nor IP filter for the incoming data-flow nor redundancy for the asset is applied, the risk-rating is considered medium.

False Positives

When the accessed target operations are not time- or resource-consuming.

Mitigation (Operations): Anti-DoS Measures

Apply anti-DoS techniques like throttling and/or per-client load blocking with quotas. Also for maintenance access routes consider applying a VPN instead of public reachable interfaces. Generally applying redundancy on the targeted technical asset reduces the risk of DoS.

ASVS Chapter: V1 - Architecture, Design and Threat Modeling Requirements
Cheat Sheet: Denial_of_Service_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **DoS-risky Access Across Trust-Boundary** was found **3 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.
Risk finding paragraphs are clickable and link to the corresponding chapter.

Low Risk Severity

Denial-of-Service risky access of **API Server** by **Browser SPA** via **HTTPS to Nginx** forwarded via **Nginx Reverse Proxy**: Exploitation likelihood is *Unlikely* with *Low* impact.
dos-risky-access-across-trust-boundary@api-server@browser-spa@browser-spa>https-to-nginx->nginx-proxy>http-to-api-server

Unchecked

Denial-of-Service risky access of **Nginx Reverse Proxy** by **Browser SPA** via **HTTPS to Nginx**: Exploitation likelihood is *Unlikely* with *Low* impact.
dos-risky-access-across-trust-boundary@nginx-proxy@browser-spa@browser-spa>https-to-nginx->

Unchecked

Denial-of-Service risky access of **PostgreSQL Database** by **API Server** via **PostgreSQL Connection**: Exploitation likelihood is *Unlikely* with *Low* impact.
dos-risky-access-across-trust-boundary@postgresql-db@api-server@api-server>postgresql-connection->

Unchecked

Unnecessary Technical Asset: 2 / 2 Risks

Description (Elevation of Privilege): CWE 1008

When a technical asset does not process any data assets, this is an indicator for an unnecessary technical asset (or for an incomplete model). This is also the case if the asset has no communication links (either outgoing or incoming).

Impact

If this risk is unmitigated, attackers might be able to target unnecessary technical assets.

Detection Logic

Technical assets not processing or storing any data assets.

Risk Rating

low

False Positives

Usually no false positives as this looks like an incomplete model.

Mitigation (Architecture): Attack Surface Reduction

Try to avoid using technical assets that do not process or store anything.

ASVS Chapter: V1 - Architecture, Design and Threat Modeling Requirements
Cheat Sheet: Attack_Surface_Analysis_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Unnecessary Technical Asset** was found **2 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.
Risk finding paragraphs are clickable and link to the corresponding chapter.

Low Risk Severity

Unnecessary Technical Asset named **AWS RDS PostgreSQL**: Exploitation likelihood is *Unlikely* with *Low* impact.
unnecessary-technical-asset@rds-postgresql

Unchecked

Unnecessary Technical Asset named **AWS S3 Notes Bucket**: Exploitation likelihood is *Unlikely* with *Low* impact.
unnecessary-technical-asset@s3-notes-bucket

Unchecked

Identified Risks by Technical Asset

In total **67 potential risks** have been identified during the threat modeling process of which **0 are rated as critical**, **2 as high**, **21 as elevated**, **38 as medium**, and **6 as low**.

These risks are distributed across **18 in-scope technical assets**. The following sub-chapters of this section describe each identified risk grouped by technical asset. The RAA value of a technical asset is the calculated "Relative Attacker Attractiveness" value in percent.

API Server: 16 / 16 Risks

Description

Node.js/Express REST API. Handles authentication (JWT/bcrypt), CRUD on notes, file uploads to MinIO. Bridges the frontend and backend networks. Single most security-critical component — compromise equals full data access.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

High Risk Severity

SQL/NoSQL-Injection risk at **API Server** against database **PostgreSQL Database** via **PostgreSQL Connection**: Exploitation likelihood is *Very Likely* with *High* impact.

sql-nosql-injection@api-server@postgresql-db@api-server>postgresql-connection

Unchecked

Elevated Risk Severity

Missing Authentication covering communication link **HTTP to API Server** from **Nginx Reverse Proxy** to **API Server**: Exploitation likelihood is *Likely* with *High* impact.

missing-authentication@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Unchecked

Missing Authentication covering communication link **Route to ECS API** from **AWS API Gateway** to **API Server**: Exploitation likelihood is *Likely* with *High* impact.

missing-authentication@aws-api-gateway>route-to-ecs-api@aws-api-gateway@api-server

Unchecked

Unencrypted Communication named **MinIO File Storage** between **API Server** and **MinIO Object Storage** transferring authentication data (like credentials, token, session-id, etc.): Exploitation likelihood is *Likely* with *High* impact.

unencrypted-communication@api-server>minio-file-storage@api-server@minio-storage

Unchecked

Unencrypted Communication named **PostgreSQL Connection** between **API Server** and **PostgreSQL Database** transferring authentication data (like credentials, token, session-id, etc.): Exploitation likelihood is *Likely* with *High* impact.

unencrypted-communication@api-server>postgresql-connection@api-server@postgresql-db

Unchecked

Unencrypted Communication named **Redis Session Store** between **API Server** and **Redis Cache** transferring authentication data (like credentials, token, session-id, etc.): Exploitation likelihood is *Likely* with *High* impact.

unencrypted-communication@api-server>redis-session-store@api-server@redis-cache

Unchecked

Path-Traversal risk at **API Server** against filesystem **MinIO Object Storage** via **MinIO File Storage**: Exploitation likelihood is *Very Likely* with *Medium* impact.

path-traversal@api-server@minio-storage@api-server>minio-file-storage

Unchecked

SQL/NoSQL-Injection risk at **API Server** against database **Redis Cache** via **Redis Session Store**: Exploitation likelihood is *Very Likely* with *Medium* impact.

sql-nosql-injection@api-server@redis-cache@api-server>redis-session-store

Unchecked

Unguarded Access from Internet of **API Server** by **AWS API Gateway** via **Route to ECS API**: Exploitation likelihood is *Very Likely* with *Medium* impact.

unguarded-access-from-internet@api-server@aws-api-gateway@aws-api-gateway>route-to-ecs-api

Unchecked

Server-Side Request Forgery (SSRF) risk at **API Server** server-side web-requesting the target **MinIO Object Storage** via **MinIO File Storage**: Exploitation likelihood is *Likely* with *Medium* impact.

server-side-request-forgery@api-server@minio-storage@api-server>minio-file-storage

Unchecked

Medium Risk Severity

Container Base Image Backdooring risk at **API Server**: Exploitation likelihood is *Unlikely* with *High* impact.

container-baseimage-backdooring@api-server

Unchecked

Missing Build Infrastructure in the threat model (referencing asset **API Server** as an example): Exploitation likelihood is *Unlikely* with *Medium* impact.

missing-build-infrastructure@api-server

Unchecked

Missing Two-Factor Authentication covering communication link **HTTP to API Server** from **Browser SPA** forwarded via **Nginx Reverse Proxy** to **API Server**: Exploitation likelihood is *Unlikely* with *Medium* impact.

missing-authentication-second-factor@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Unchecked

Missing Vault (Secret Storage) in the threat model (referencing asset **API Server** as an example): Exploitation likelihood is *Unlikely* with *Medium* impact.

missing-vault@api-server

Unchecked

Missing Web Application Firewall (WAF) risk at **API Server**: Exploitation likelihood is *Unlikely* with *Medium* impact.

missing-waf@api-server

Unchecked

Low Risk Severity

Denial-of-Service risky access of **API Server** by **Browser SPA** via **HTTPS to Nginx** forwarded via **Nginx Reverse Proxy**: Exploitation likelihood is *Unlikely* with *Low* impact.

dos-risky-access-across-trust-boundary@api-server@browser-spa@browser-spa>https-to-nginx->nginx-proxy>http-to-api-server

Unchecked

Asset Information

ID:	api-server
Type:	process
Usage:	business
RAA:	44 %
Size:	service
Technology:	web-service-rest
Tags:	express, jwt, nodejs, trike-trust-level-admin
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	true
Client by Human:	false
Data Processed:	File Attachments, Note Content, Session Tokens, User Credentials
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team
Confidentiality:	strictly-confidential (rated 5 in scale of 5)

Integrity:	critical	(rated 4 in scale of 5)
Availability:	critical	(rated 4 in scale of 5)
CIA-Justification:	The API processes all user data. It bridges both Docker networks — a compromised API container can pivot into the internal data tier.	

Outgoing Communication Links: 3

Target technical asset names are clickable and link to the corresponding chapter.

Redis Session Store (outgoing)

Session token storage and JWT revocation list. Password-protected (requirepass) but no TLS.

Target:	Redis Cache
Protocol:	nosql-access-protocol
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	Session Tokens
Data Received:	Session Tokens

PostgreSQL Connection (outgoing)

Application-level SQL queries for user auth, notes CRUD. Uses parameterized queries (pg library) to prevent SQL injection. No TLS on the connection — traffic is plaintext within backend-net.

Target:	PostgreSQL Database
Protocol:	sql-access-protocol
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false

Data Sent:	Note Content, User Credentials
Data Received:	Note Content, User Credentials

MinIO File Storage (outgoing)

INTENTIONAL MISCONFIGURATION: S3 API calls to MinIO over plain HTTP. Default minioadmin/minioadmin credentials are hardcoded in compose. Any container on backend-net can read all stored files.

Target:	MinIO Object Storage
Protocol:	http
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	intentional-misconfiguration
VPN:	false
IP-Filtered:	false
Data Sent:	File Attachments
Data Received:	File Attachments

Incoming Communication Links: 2

Source technical asset names are clickable and link to the corresponding chapter.

HTTP to API Server (incoming)

INTENTIONAL MISCONFIGURATION: Traffic between Nginx and the API server uses plaintext HTTP. An attacker with access to the Docker network (container escape, compromised sidecar) can intercept credentials, session tokens, and note content in transit.

Source:	Nginx Reverse Proxy
Protocol:	http
Encrypted:	false
Authentication:	none
Authorization:	none
Read-Only:	false
Usage:	business
Tags:	intentional-misconfiguration
VPN:	false
IP-Filtered:	false

Data Received:	File Attachments, Note Content, User Credentials
Data Sent:	File Attachments, Note Content

Route to ECS API (incoming)

HTTP forward from API Gateway to the ECS-hosted API containers via ALB. Traffic is within the AWS VPC — no public routing for backend traffic.

Source:	AWS API Gateway
Protocol:	https
Encrypted:	true
Authentication:	none
Authorization:	none
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	true
Data Received:	File Attachments, Note Content, User Credentials
Data Sent:	File Attachments, Note Content

MinIO Object Storage: 4 / 4 Risks

Description

S3-compatible object store for file attachments. INTENTIONAL MISCONFIGURATION: Uses default minioadmin/minioadmin credentials (visible in docker-compose.yml) and stores files without encryption at rest. HTTP-only (no TLS) within the Docker network.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

High Risk Severity

Exposed Default Credentials: **MinIO Object Storage** is tagged as intentional-misconfiguration and stores confidential data: Exploitation likelihood is *Very Likely* with *High* impact.

exposed-default-credentials@minio-storage

Unchecked

Medium Risk Severity

Container Base Image Backdooring risk at **MinIO Object Storage**: Exploitation likelihood is *Unlikely* with *Medium* impact.

container-baseimage-backdooring@minio-storage

Unchecked

Unguarded Direct Datastore Access of **MinIO Object Storage** by **API Server** via **MinIO File Storage**: Exploitation likelihood is *Likely* with *Low* impact.

unguarded-direct-datastore-access@api-server>minio-file-storage@api-server@minio-storage

Unchecked

Unencrypted Technical Asset named **MinIO Object Storage**: Exploitation likelihood is *Unlikely* with *Medium* impact.

unencrypted-asset@minio-storage

In Progress 2026-04-18 Security Team VAULT-110
MinIO server-side encryption (SSE-S3) is being evaluated. Default minioadmin credentials will be rotated before production deployment. This finding is a CRITICAL blocker for launch. Tracked in VAULT-110.

Asset Information

ID:	minio-storage
Type:	datastore
Usage:	business
RAA:	21 %

Size:	service
Technology:	object-storage
Tags:	criticality-high, information-asset-container, intentional-misconfiguration, minio, object-storage, s3, third-party-shared
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	File Attachments
Data Stored:	File Attachments
Formats Accepted:	File

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Stores user file uploads which may contain sensitive documents. Default credentials + no encryption = trivial exfiltration by any process on backend-net.	

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

MinIO File Storage (incoming)

INTENTIONAL MISCONFIGURATION: S3 API calls to MinIO over plain HTTP. Default minioadmin/minioadmin credentials are hardcoded in compose. Any container on backend-net can read all stored files.

Source:	API Server
Protocol:	http
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user

Read-Only:	false
Usage:	business
Tags:	intentional-misconfiguration
VPN:	false
IP-Filtered:	false
Data Received:	File Attachments
Data Sent:	File Attachments

AWS RDS PostgreSQL: 3 / 3 Risks

Description

AWS RDS PostgreSQL instance (production). Replaces the Docker-based PostgreSQL in production deployments. Automated backups enabled (7-day retention). INTENTIONAL: Encryption at rest is disabled (storage_encrypted = false in Terraform). Public accessibility is off but subnet group is in app subnet (not data subnet), creating potential exposure via compromised app instances.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Missing Hardening risk at **AWS RDS PostgreSQL**: Exploitation likelihood is *Likely* with *Medium* impact.

missing-hardening@rds-postgresql

Unchecked

Medium Risk Severity

Unencrypted Technical Asset named **AWS RDS PostgreSQL** missing end user individual encryption with data-with-end-user-individual-key: Exploitation likelihood is *Unlikely* with *High* impact.

unencrypted-asset@rds-postgresql

Unchecked

Low Risk Severity

Unnecessary Technical Asset named **AWS RDS PostgreSQL**: Exploitation likelihood is *Unlikely* with *Low* impact.

unnecessary-technical-asset@rds-postgresql

Unchecked

Asset Information

ID:	rds-postgresql
Type:	datastore
Usage:	business
RAA:	54 %
Size:	service

Technology:	managed-database
Tags:	aws, cloud-native, criticality-high, has-automated-backup, information-asset-container, postgresql, rds
Internet:	false
Machine:	virtual
Encryption:	none
Multi-Tenant:	false
Redundant:	true
Custom-Developed:	false
Client by Human:	false
Data Processed:	Note Content, User Credentials
Data Stored:	Note Content, User Credentials
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	strictly-confidential	(rated 5 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	critical	(rated 4 in scale of 5)
CIA-Justification:	Production database stores all user PII and note content. Encryption disabled means RDS snapshot exfiltration exposes all data in plaintext. Snapshot sharing misconfiguration (a common AWS incident) bypasses all access controls.	

ECS Container Platform: 4 / 4 Risks

Description

AWS ECS (Fargate) running the API and Nginx containers in production. INTENTIONAL: ECR Enhanced Scanning (Inspector v2) is not enabled. Running containers are not scanned post-deployment for newly disclosed CVEs. Container images are pulled from ECR without signature verification.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Missing Hardening risk at **ECS Container Platform**: Exploitation likelihood is *Likely* with *Medium* impact.

missing-hardening@ecs-platform

Unchecked

Path-Traversal risk at **ECS Container Platform** against filesystem **AWS ECR Container Registry** via **Pull Images from ECR**: Exploitation likelihood is *Likely* with *Medium* impact.

path-traversal@ecs-platform@container-registry@ecs-platform>pull-images-from-ecr

Unchecked

Medium Risk Severity

Container Platform Escape risk at **ECS Container Platform**: Exploitation likelihood is *Unlikely* with *High* impact.

container-platform-escape@ecs-platform

Unchecked

Server-Side Request Forgery (SSRF) risk at **ECS Container Platform** server-side web-requesting the target **AWS ECR Container Registry** via **Pull Images from ECR**: Exploitation likelihood is *Unlikely* with *Medium* impact.

server-side-request-forgery@ecs-platform@container-registry@ecs-platform>pull-images-from-ecr

Unchecked

Asset Information

ID:	ecs-platform
Type:	process
Usage:	business
RAA:	100 %

Size:	service
Technology:	container-platform
Tags:	aws, cloud-native, ecs, fargate
Internet:	false
Machine:	virtual
Encryption:	transparent
Multi-Tenant:	false
Redundant:	true
Custom-Developed:	false
Client by Human:	false
Data Processed:	Build Artifacts, Note Content, User Credentials
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	strictly-confidential	(rated 5 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	critical	(rated 4 in scale of 5)
CIA-Justification:	ECS runs the production API containers. Container escape or image tampering grants access to the application runtime and its mounted secrets. No image scanning means known CVEs in base images are undetected until exploitation.	

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

Pull Images from ECR (outgoing)

ECS task pulls Docker images from ECR at task launch. Images are not verified against signatures before execution.

Target:	AWS ECR Container Registry
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	true

Usage:	devops
Tags:	none
VPN:	false
IP-Filtered:	true
Data Sent:	none
Data Received:	Build Artifacts

GitHub Actions Build Pipeline: 5 / 5 Risks

Description

GitHub Actions CI/CD pipeline. Builds Docker images, runs tests, pushes to ECR. INTENTIONAL: No SBOM generation (no Syft or CycloneDX step configured). No build provenance attestation (no SLSA provenance step). Hardcoded AWS_SECRET_ACCESS_KEY in workflow environment variables.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Path-Traversal risk at **GitHub Actions Build Pipeline** against filesystem **AWS ECR Container Registry** via **Push Image to Registry**: Exploitation likelihood is *Likely* with *Medium* impact.

path-traversal@build-pipeline@container-registry@build-pipeline>push-image-to-registry

Unchecked

Medium Risk Severity

Code Backdooring risk at **GitHub Actions Build Pipeline**: Exploitation likelihood is *Unlikely* with *High* impact.

code-backdooring@build-pipeline

Unchecked

Server-Side Request Forgery (SSRF) risk at **GitHub Actions Build Pipeline** server-side web-requesting the target **AWS ECR Container Registry** via **Push Image to Registry**: Exploitation likelihood is *Unlikely* with *Medium* impact.

server-side-request-forgery@build-pipeline@container-registry@build-pipeline>push-image-to-registry

Unchecked

Unchecked Deployment risk at **GitHub Actions Build Pipeline**: Exploitation likelihood is *Unlikely* with *Medium* impact.

unchecked-deployment@build-pipeline

Unchecked

Unguarded Access from Internet of **GitHub Actions Build Pipeline** by **VaultNote Source Repository** via **Push to Pipeline**: Exploitation likelihood is *Very Likely* with *Low* impact.

unguarded-access-from-internet@build-pipeline@source-repo@source-repo>push-to-pipeline

Unchecked

Asset Information

ID:	build-pipeline
Type:	process
Usage:	devops
RAA:	12 %
Size:	service
Technology:	build-pipeline
Tags:	ci, github-actions, supply-chain
Internet:	true
Machine:	virtual
Encryption:	transparent
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	Build Artifacts
Data Stored:	Build Artifacts
Formats Accepted:	File

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Build pipeline is a privileged component — it has read access to secrets and write access to the container registry. Compromise = full supply chain injection. No SBOM means post-build vulnerability tracking is impossible.	

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

Push Image to Registry (outgoing)

docker push to AWS ECR after build completes. Images are tagged with the git commit SHA and pushed without cryptographic signing (no Cosign step).

Target:	AWS ECR Container Registry

Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	devops
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	Build Artifacts
Data Received:	none

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

Push to Pipeline (incoming)

Git push triggers GitHub Actions workflow. The webhook carries the commit SHA and branch reference to the CI runner.

Source:	VaultNote Source Repository
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	devops
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	Build Artifacts
Data Sent:	none

Lambda Email Notifier: 1 / 1 Risk

Description

AWS Lambda function that sends transactional emails (note-share invitations, account verification) triggered by SQS messages from the API server. Processes user email addresses (PII) and note titles in notification payloads. INTENTIONAL: No dead letter queue (DLQ) configured on the SQS event source. Failed notifications are silently dropped after 3 retries with no audit trail.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Server-Side Request Forgery (SSRF) risk at **Lambda Email Notifier** server-side web-requesting the target **AWS SES** via **SES Email Send**: Exploitation likelihood is *Likely* with *Medium* impact.

server-side-request-forgery@lambda-notifier@aws-ses@lambda-notifier>ses-email-send

Unchecked

Asset Information

ID:	lambda-notifier
Type:	process
Usage:	business
RAA:	2 %
Size:	component
Technology:	serverless-function
Tags:	aws, cloud-native, lambda, notifications
Internet:	false
Machine:	virtual
Encryption:	transparent
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	true
Client by Human:	false
Data Processed:	Notification Payloads
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	operational	(rated 2 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Lambda processes email addresses and note titles. PII loss due to silent failure creates compliance issues (incomplete processing records). Overpermissive IAM role could be exploited to access other AWS resources.	

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

SES Email Send (outgoing)

Lambda calls AWS SES to deliver transactional emails. IAM role grants ses:SendEmail only; no SendRawEmail permission. No SES sending rate limit configured on the IAM policy — a compromised Lambda could exhaust the SES daily quota.

Target:	AWS SES
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	Notification Payloads
Data Received:	none

Nginx Reverse Proxy: 6 / 6 Risks

Description

Nginx terminates TLS and forwards all API traffic to the Express server over plain HTTP (port 3000). This is the DMZ entry point.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Lateral Movement via Shared Runtime on **Docker Host**: assets from 3 trust zones co-located: Exploitation likelihood is *Likely* with *High* impact.

lateral-movement-shared-runtime@docker-host

Unchecked

Missing Content-Security-Policy: **Nginx Reverse Proxy** serves browser clients handling auth tokens without a verified CSP header: Exploitation likelihood is *Likely* with *Medium* impact.

missing-csp-header@nginx-proxy

Unchecked

Unencrypted Communication named **HTTP to API Server** between **Nginx Reverse Proxy** and **API Server**: Exploitation likelihood is *Likely* with *Medium* impact.

unencrypted-communication@nginx-proxy>http-to-api-server@nginx-proxy@api-server

In Progress 2026-04-18 Security Team VAULT-102
Migrating Nginx-to-API communication to mTLS. A separate CA will be provisioned inside the Docker network. PR #47 is in review; target completion sprint 22.

Medium Risk Severity

Container Base Image Backdooring risk at **Nginx Reverse Proxy**: Exploitation likelihood is *Unlikely* with *High* impact.

container-baseimage-backdooring@nginx-proxy

Unchecked

Missing Identity Store in the threat model (referencing asset **Nginx Reverse Proxy** as an example): Exploitation likelihood is *Unlikely* with *Medium* impact.

missing-identity-store@nginx-proxy

Unchecked

Low Risk Severity

Denial-of-Service risky access of **Nginx Reverse Proxy** by **Browser SPA** via **HTTPS to Nginx**: Exploitation likelihood is *Unlikely* with *Low* impact.

dos-risky-access-across-trust-boundary@nginx-proxy@browser-spa@browser-spa>https-to-nginx->

Unchecked

Asset Information

ID:	nginx-proxy
Type:	process
Usage:	business
RAA:	1 %
Size:	service
Technology:	reverse-proxy
Tags:	nginx, tls-termination
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	File Attachments, Note Content, User Credentials
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	strictly-confidential	(rated 5 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	critical	(rated 4 in scale of 5)
CIA-Justification:	Nginx is the single external entry point. Its compromise exposes all downstream services. Availability is critical — outage = full service outage.	

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

HTTP to API Server (outgoing)

INTENTIONAL MISCONFIGURATION: Traffic between Nginx and the API server uses plaintext HTTP. An attacker with access to the Docker network (container escape, compromised sidecar) can intercept credentials, session tokens, and note content in transit.

Target:	API Server
Protocol:	http
Encrypted:	false
Authentication:	none
Authorization:	none
Read-Only:	false
Usage:	business
Tags:	intentional-misconfiguration
VPN:	false
IP-Filtered:	false
Data Sent:	File Attachments, Note Content, User Credentials
Data Received:	File Attachments, Note Content

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

HTTPS to Nginx (incoming)

User-initiated HTTPS requests carrying credentials (login) and note content (create/edit). This is the only encrypted link in the chain.

Source:	Browser SPA
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	end-user-identity-propagation
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	File Attachments, Note Content, User Credentials
Data Sent:	File Attachments, Note Content

PostgreSQL Database: 5 / 5 Risks

Description

Primary relational datastore. Contains users table (email + bcrypt password hash), notes table (title + content), and attachments metadata. INTENTIONAL MISCONFIGURATION: No encryption at rest (encryption: none).

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Missing Hardening risk at **PostgreSQL Database**: Exploitation likelihood is *Likely* with *Medium* impact.

missing-hardening@postgresql-db

Unchecked

Unguarded Direct Datastore Access of **PostgreSQL Database** by **API Server** via **PostgreSQL Connection**: Exploitation likelihood is *Likely* with *Medium* impact.

unguarded-direct-datastore-access@api-server>postgresql-connection@api-server@postgresql-db

Unchecked

Medium Risk Severity

Container Base Image Backdooring risk at **PostgreSQL Database**: Exploitation likelihood is *Unlikely* with *High* impact.

container-baseimage-backdooring@postgresql-db

Unchecked

Unencrypted Technical Asset named **PostgreSQL Database** missing end user individual encryption with data-with-end-user-individual-key: Exploitation likelihood is *Unlikely* with *High* impact.

unencrypted-asset@postgresql-db

Accepted 2026-04-15 Infrastructure Team VAULT-89

The PostgreSQL volume runs on a host-level LUKS-encrypted partition in all production deployments. The Threagile finding is acknowledged as informational for this demo environment which uses unencrypted storage. Production database TDE via pgcrypto will be evaluated in Q3.

Low Risk Severity

Denial-of-Service risky access of **PostgreSQL Database** by **API Server** via **PostgreSQL Connection**: Exploitation likelihood is *Unlikely* with *Low* impact.

dos-risky-access-across-trust-boundary@postgresql-db@api-server@api-server>postgresql-connection->

Unchecked

Asset Information

ID:	postgresql-db
Type:	datastore
Usage:	business
RAA:	54 %
Size:	service
Technology:	database
Tags:	criticality-high, information-asset-container, postgresql, relational, third-party-shared
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	Note Content, User Credentials
Data Stored:	Note Content, User Credentials
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	strictly-confidential	(rated 5 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	critical	(rated 4 in scale of 5)
CIA-Justification:	Holds all user PII (email, hashed passwords) and all note content. Strictly confidential — unencrypted disk access exposes all data.	

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

PostgreSQL Connection (incoming)

Application-level SQL queries for user auth, notes CRUD. Uses parameterized queries (pg library) to prevent SQL injection. No TLS on the connection — traffic is plaintext within backend-net.

Source:	API Server
Protocol:	sql-access-protocol
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	Note Content, User Credentials
Data Sent:	Note Content, User Credentials

AWS ECR Container Registry: 3 / 3 Risks

Description

AWS Elastic Container Registry storing VaultNote Docker images (api, nginx). INTENTIONAL: Image vulnerability scanning is disabled (ECR Basic scanning only, not Enhanced/Inspector). Images are not signed — no Cosign/Sigstore policy enforced. Repository is private but no admission controller validates signatures at deploy time.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Code Backdooring risk at **AWS ECR Container Registry**: Exploitation likelihood is *Unlikely* with *High* impact.

code-backdooring@container-registry

Unchecked

Unguarded Access from Internet of **AWS ECR Container Registry** by **GitHub Actions Build Pipeline** via **Push Image to Registry**: Exploitation likelihood is *Very Likely* with *Low* impact.

unguarded-access-from-internet@container-registry@build-pipeline@build-pipeline>push-image-to-registry

Unchecked

Low Risk Severity

Unchecked Deployment risk at **AWS ECR Container Registry**: Exploitation likelihood is *Unlikely* with *Low* impact.

unchecked-deployment@container-registry

Unchecked

Asset Information

ID:	container-registry
Type:	datastore
Usage:	devops
RAA:	9 %
Size:	service
Technology:	container-registry
Tags:	aws, ecr, supply-chain
Internet:	true

Machine:	virtual
Encryption:	transparent
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	Build Artifacts
Data Stored:	Build Artifacts
Formats Accepted:	File

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Container registry is the final source of truth for deployed images. Tampering with registry contents (image swap or layer injection) propagates malicious code to every deployment. No signing verification means a registry compromise is undetectable until damage is done.	

Incoming Communication Links: 2

Source technical asset names are clickable and link to the corresponding chapter.

Pull Images from ECR (incoming)

ECS task pulls Docker images from ECR at task launch. Images are not verified against signatures before execution.

Source:	ECS Container Platform
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	true
Usage:	devops
Tags:	none
VPN:	false
IP-Filtered:	true

Data Received:	none
Data Sent:	Build Artifacts

Push Image to Registry (incoming)

docker push to AWS ECR after build completes. Images are tagged with the git commit SHA and pushed without cryptographic signing (no Cosign step).

Source:	GitHub Actions Build Pipeline
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	devops
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	Build Artifacts
Data Sent:	none

AWS S3 Notes Bucket: 2 / 2 Risks

Description

AWS S3 bucket storing file attachments for production deployments. Replaces MinIO S3-compatible storage in the cloud environment. Bucket is private (Block Public Access enabled) but server-side encryption is not configured. INTENTIONAL: SSE-S3 or SSE-KMS not enabled. Objects are stored unencrypted at the storage layer — provider-side access exposes raw data.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Unencrypted Technical Asset named **AWS S3 Notes Bucket**: Exploitation likelihood is *Unlikely* with *Medium* impact.

unencrypted-asset@s3-notes-bucket

Unchecked

Low Risk Severity

Unnecessary Technical Asset named **AWS S3 Notes Bucket**: Exploitation likelihood is *Unlikely* with *Low* impact.

unnecessary-technical-asset@s3-notes-bucket

Unchecked

Asset Information

ID:	s3-notes-bucket
Type:	datastore
Usage:	business
RAA:	21 %
Size:	service
Technology:	object-storage
Tags:	aws, cloud-native, criticality-high, information-asset-container, s3, third-party-shared
Internet:	false
Machine:	virtual
Encryption:	none
Multi-Tenant:	false
Redundant:	true

Custom-Developed:	false
Client by Human:	false
Data Processed:	File Attachments
Data Stored:	File Attachments
Formats Accepted:	File

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	S3 stores user-uploaded files which may contain sensitive documents. No SSE means provider-side access (support channel, storage engineer) reads raw data. third-party-shared because S3 is operated by AWS and subject to subprocessor data protection requirements. No DPA review completed.	

LLM Note Summarizer: 2 / 2 Risks

Description

OpenAI-compatible LLM inference endpoint used to generate AI summaries of user notes on demand. Deployed as a containerised model proxy (LiteLLM) with a public API endpoint for testing. INTENTIONAL: Endpoint is internet-accessible and lacks authentication — the has-authentication tag is intentionally absent to demonstrate the finding. No adversarial input shield (Llama Guard or guardrails) is configured. System prompt contains internal business context visible to adversarial probing.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Server-Side Request Forgery (SSRF) risk at **LLM Note Summarizer** server-side web-requesting the target **Note RAG Pipeline** via **Call RAG Pipeline**: Exploitation likelihood is *Likely* with *Low* impact.

server-side-request-forgery@llm-summarizer@rag-pipeline@llm-summarizer>call-rag-pipeline

Unchecked

Server-Side Request Forgery (SSRF) risk at **LLM Note Summarizer** server-side web-requesting the target **VaultNote Vector Store** via **Query Vector Store**: Exploitation likelihood is *Likely* with *Low* impact.

server-side-request-forgery@llm-summarizer@vector-store@llm-summarizer>query-vector-store

Unchecked

Asset Information

ID:	llm-summarizer
Type:	process
Usage:	business
RAA:	20 %
Size:	service
Technology:	llm-inference-endpoint
Tags:	ai, cloud-native, llm, openai
Internet:	true
Machine:	virtual
Encryption:	transparent
Multi-Tenant:	false
Redundant:	false

Custom-Developed:	true
Client by Human:	false
Data Processed:	AI Model Responses, Embedding Vectors, Note Content
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	LLM endpoint processes user note content. Unauthenticated internet exposure allows arbitrary prompt submission, system prompt extraction via jailbreaks, and inference cost abuse. No guardrails means malicious inputs could generate harmful content or exfiltrate context window data.	

Outgoing Communication Links: 2

Target technical asset names are clickable and link to the corresponding chapter.

Query Vector Store (outgoing)

LLM retrieves relevant note chunks from the vector store to build RAG context. Similarity search over user-owned embeddings.

Target:	VaultNote Vector Store
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	true
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	Embedding Vectors
Data Received:	Note Content

Call RAG Pipeline (outgoing)

LLM inference endpoint triggers the RAG pipeline to build context from stored note embeddings before generating a summary response.

Target:	Note RAG Pipeline
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	Note Content
Data Received:	Note Content

Note RAG Pipeline: 3 / 3 Risks

Description

Retrieval-Augmented Generation pipeline that assembles context chunks from the vector store before forwarding to the LLM. Built on LangChain. INTENTIONAL: No input validation or injection guard on retrieved chunks (has-input-validation absent). A note containing prompt injection payloads could be retrieved as context and alter LLM behavior. No retrieval filtering — all results from the vector store are injected into context without relevance threshold or content sanitisation.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Container Base Image Backdooring risk at **Note RAG Pipeline**: Exploitation likelihood is *Unlikely* with *Medium* impact.

container-baseimage-backdooring@rag-pipeline

Unchecked

Unguarded Access from Internet of **Note RAG Pipeline** by **LLM Note Summarizer** via **Call RAG Pipeline**: Exploitation likelihood is *Very Likely* with *Low* impact.

unguarded-access-from-internet@rag-pipeline@llm-summarizer@llm-summarizer>call-rag-pipeline

Unchecked

Server-Side Request Forgery (SSRF) risk at **Note RAG Pipeline** server-side web-requesting the target **VaultNote Vector Store** via **Retrieve from Vector Store**: Exploitation likelihood is *Likely* with *Low* impact.

server-side-request-forgery@rag-pipeline@vector-store@rag-pipeline>retrieve-from-vector-store

Unchecked

Asset Information

ID:	rag-pipeline
Type:	process
Usage:	business
RAA:	11 %
Size:	component
Technology:	rag-pipeline
Tags:	ai, langchain, rag
Internet:	false
Machine:	container
Encryption:	transparent

Multi-Tenant:	false
Redundant:	false
Custom-Developed:	true
Client by Human:	false
Data Processed:	Embedding Vectors, Note Content
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	RAG pipeline is the injection point between user-controlled note content and the LLM context window. Without input validation, a malicious note can manipulate the LLM into ignoring instructions, leaking system prompts, or generating harmful content on behalf of an attacker.	

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

Retrieve from Vector Store (outgoing)

RAG pipeline queries the vector store for nearest-neighbor embeddings to build context chunks. Returns raw embedding text — no content filtering before injection into LLM context.

Target:	VaultNote Vector Store
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	true
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	Embedding Vectors
Data Received:	Note Content

Incoming Communication Links: 1
Source technical asset names are clickable and link to the corresponding chapter.

Call RAG Pipeline (incoming)
LLM inference endpoint triggers the RAG pipeline to build context from stored note embeddings before generating a summary response.

Source:	LLM Note Summarizer
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	Note Content
Data Sent:	Note Content

Redis Cache: 3 / 3 Risks

Description

In-memory store used as session store and JWT revocation list. INTENTIONAL MISCONFIGURATION: No encryption at rest. If Redis persistence is enabled and the RDB file is exfiltrated, session tokens are exposed.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Container Base Image Backdooring risk at **Redis Cache**: Exploitation likelihood is *Unlikely* with *Medium* impact.

container-baseimage-backdooring@redis-cache

Unchecked

Unencrypted Technical Asset named **Redis Cache**: Exploitation likelihood is *Unlikely* with *Medium* impact.

unencrypted-asset@redis-cache

Unchecked

Unguarded Direct Datastore Access of **Redis Cache** by **API Server** via **Redis Session Store**: Exploitation likelihood is *Likely* with *Low* impact.

unguarded-direct-datastore-access@api-server>redis-session-store@api-server@redis-cache

Unchecked

Asset Information

ID:	redis-cache
Type:	datastore
Usage:	business
RAA:	22 %
Size:	component
Technology:	database
Tags:	cache, criticality-high, information-asset-container, redis, session-store
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false

Client by Human:	false
Data Processed:	Session Tokens
Data Stored:	Session Tokens
Formats Accepted:	none of the special data formats accepted

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	important	(rated 3 in scale of 5)
CIA-Justification:	Session token exposure enables account takeover. Non-persistent by default but the container volume could persist RDB snapshots.	

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

Redis Session Store (incoming)

Session token storage and JWT revocation list. Password-protected (requirepass) but no TLS.

Source:	API Server
Protocol:	nosql-access-protocol
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	Session Tokens
Data Sent:	Session Tokens

VaultNote Source Repository: 4 / 4 Risks

Description

GitHub repository hosting the VaultNote monorepo (API, frontend, Terraform). Contains application secrets in compose files (minioadmin credentials visible in git history) and Terraform state references. INTENTIONAL: No branch protection rules configured — developers push directly to main, bypassing review and signed-commit requirements.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Code Backdooring risk at **VaultNote Source Repository**: Exploitation likelihood is *Unlikely* with *High* impact.

code-backdooring@source-repo

Unchecked

Accidental Secret Leak(Git) risk at **VaultNote Source Repository**: Git Leak Prevention: Exploitation likelihood is *Unlikely* with *Medium* impact.

accidental-secret-leak@source-repo

Unchecked

Server-Side Request Forgery (SSRF) risk at **VaultNote Source Repository** server-side web-requesting the target **GitHub Actions Build Pipeline** via **Push to Pipeline**: Exploitation likelihood is *Unlikely* with *Medium* impact.

server-side-request-forgery@source-repo@build-pipeline@source-repo>push-to-pipeline

Unchecked

Unchecked Deployment risk at **VaultNote Source Repository**: Exploitation likelihood is *Unlikely* with *Medium* impact.

unchecked-deployment@source-repo

Unchecked

Asset Information

ID:	source-repo
Type:	process
Usage:	business
RAA:	12 %
Size:	service
Technology:	sourcecode-repository
Tags:	git, github, supply-chain

Internet:	true
Machine:	virtual
Encryption:	transparent
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	Build Artifacts
Data Stored:	Build Artifacts
Formats Accepted:	File

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Source code is the root of the supply chain. Integrity is critical — malicious code injected here propagates to every built artifact. May contain leaked secrets.	

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

Push to Pipeline (outgoing)

Git push triggers GitHub Actions workflow. The webhook carries the commit SHA and branch reference to the CI runner.

Target:	GitHub Actions Build Pipeline
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	devops
Tags:	none
VPN:	false
IP-Filtered:	false

Data Sent: Build Artifacts
Data Received: none

VaultNote Vector Store: 2 / 2 Risks

Description

Qdrant vector database storing embedding vectors for semantic search and RAG context retrieval. Vectors are generated by the embedding service from user note content. INTENTIONAL: No encryption at rest configured (encryption: none). The vector store is queryable by the LLM pipeline and the API server without row-level isolation — all user embeddings share the same collection. No access control list separates per-user embedding queries.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Container Base Image Backdooring risk at **VaultNote Vector Store**: Exploitation likelihood is *Unlikely* with *Medium* impact.

container-baseimage-backdooring@vector-store

Unchecked

Unguarded Access from Internet of **VaultNote Vector Store** by **LLM Note Summarizer** via **Query Vector Store**: Exploitation likelihood is *Very Likely* with *Low* impact.

unguarded-access-from-internet@vector-store@llm-summarizer@llm-summarizer>query-vector-store

Unchecked

Asset Information

ID:	vector-store
Type:	datastore
Usage:	business
RAA:	26 %
Size:	service
Technology:	vector-store
Tags:	ai, criticality-high, embeddings, information-asset-container, qdrant
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	Embedding Vectors, Note Content

Data Stored: Embedding Vectors
Formats Accepted: JSON

Asset Rating

Owner: VaultNote Team
Confidentiality: confidential (rated 4 in scale of 5)
Integrity: operational (rated 2 in scale of 5)
Availability: operational (rated 2 in scale of 5)
CIA-Justification: Embedding vectors encode semantic information from user notes. Even without raw text, vectors can be used to infer content via inversion attacks. Unencrypted storage and lack of per-user access control create cross-tenant leakage risk.

Incoming Communication Links: 2

Source technical asset names are clickable and link to the corresponding chapter.

Retrieve from Vector Store (incoming)
RAG pipeline queries the vector store for nearest-neighbor embeddings to build context chunks. Returns raw embedding text — no content filtering before injection into LLM context.

Source: Note RAG Pipeline
Protocol: https
Encrypted: true
Authentication: credentials
Authorization: technical-user
Read-Only: true
Usage: business
Tags: none
VPN: false
IP-Filtered: false
Data Received: Embedding Vectors
Data Sent: Note Content

Query Vector Store (incoming)
LLM retrieves relevant note chunks from the vector store to build RAG context. Similarity search over user-owned embeddings.

Source: LLM Note Summarizer

Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	true
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	Embedding Vectors
Data Sent:	Note Content

AWS API Gateway: 0 / 0 Risks

Description

AWS API Gateway (HTTP API) fronting the ECS API containers in production. Routes traffic from the ALB to backend ECS tasks. INTENTIONAL: No usage plan or throttling policy configured. No WAF association on the API gateway stage. Auth relies entirely on the API server (JWT) — no gateway-level auth enforcement.

Identified Risks of Asset

No risksStr were identified.

Asset Information

ID:	aws-api-gateway
Type:	process
Usage:	business
RAA:	30 %
Size:	service
Technology:	cloud-api-gateway
Tags:	api-gateway, aws, cloud-native
Internet:	true
Machine:	virtual
Encryption:	transparent
Multi-Tenant:	false
Redundant:	true
Custom-Developed:	false
Client by Human:	false
Data Processed:	File Attachments, Note Content, User Credentials
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	strictly-confidential	(rated 5 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	critical	(rated 4 in scale of 5)

CIA-Justification: API gateway is the single internet entry point in production. No rate limiting = credential stuffing and cost amplification risk. WAF absence leaves XSS/SQLi traffic unfiltered before it reaches the API.

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

Route to ECS API (outgoing)

HTTP forward from API Gateway to the ECS-hosted API containers via ALB. Traffic is within the AWS VPC — no public routing for backend traffic.

Target:	API Server
Protocol:	https
Encrypted:	true
Authentication:	none
Authorization:	none
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	true
Data Sent:	File Attachments, Note Content, User Credentials
Data Received:	File Attachments, Note Content

AWS SES: 0 / 0 Risks

Description

AWS Simple Email Service used for transactional email delivery (note-share invitations, account verification). Managed service — not custom-developed. SES sending identity (domain) is verified; DKIM and SPF records are configured. No dedicated sending quota alarm; a compromised Lambda could exhaust the daily send limit and cause notification delivery failure.

Identified Risks of Asset

No risksStr were identified.

Asset Information

ID:	aws-ses
Type:	process
Usage:	business
RAA:	0 %
Size:	service
Technology:	mail-server
Tags:	aws, cloud-native, email, ses
Internet:	true
Machine:	virtual
Encryption:	transparent
Multi-Tenant:	false
Redundant:	true
Custom-Developed:	false
Client by Human:	false
Data Processed:	Notification Payloads
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	operational	(rated 2 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)

CIA-Justification: SES handles PII (email addresses). Sending quota exhaustion prevents delivery of account verification emails, blocking user registration. Email spoofing via insufficient SPF/DKIM/DMARC alignment is possible if domain records are misconfigured.

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

SES Email Send (incoming)

Lambda calls AWS SES to deliver transactional emails. IAM role grants ses:SendEmail only; no SendRawEmail permission. No SES sending rate limit configured on the IAM policy — a compromised Lambda could exhaust the SES daily quota.

Source:	Lambda Email Notifier
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	Notification Payloads
Data Sent:	none

Browser SPA: 0 / 0 Risks

Description

React Single Page Application served from Nginx. Users authenticate, manage notes, and upload files through this interface.

Identified Risks of Asset

No risksStr were identified.

Asset Information

ID:	browser-spa
Type:	external-entity
Usage:	business
RAA:	29 %
Size:	application
Technology:	browser
Tags:	react, spa, trike-actor, trike-actor-untrusted
Internet:	true
Machine:	physical
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	true
Client by Human:	true
Data Processed:	File Attachments, Note Content, User Credentials
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	End Users	
Confidentiality:	strictly-confidential	(rated 5 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Client-side code is public; the trust boundary here is user data entered into the browser. XSS would allow an attacker to exfiltrate session tokens.	

Outgoing Communication Links: 1
Target technical asset names are clickable and link to the corresponding chapter.

HTTPS to Nginx (outgoing)
User-initiated HTTPS requests carrying credentials (login) and note content (create/edit). This is the only encrypted link in the chain.

Target:	Nginx Reverse Proxy
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	end-user-identity-propagation
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	File Attachments, Note Content, User Credentials
Data Received:	File Attachments, Note Content

Identified Data Breach Probabilities by Data Asset

In total **67 potential risks** have been identified during the threat modeling process of which **0 are rated as critical**, **2 as high**, **21 as elevated**, **38 as medium**, and **6 as low**.

These risks are distributed across **8 data assets**. The following sub-chapters of this section describe the derived data breach probabilities grouped by data asset.

Technical asset names and risk IDs are clickable and link to the corresponding chapter.

Build Artifacts: 17 / 17 Risks

Docker container images, compiled source bundles, and dependency lockfiles produced by the CI/CD pipeline. Integrity is critical — these are the deployable units of the application.

ID:	build-artifacts	
Usage:	devops	
Quantity:	few	
Tags:	supply-chain	
Origin:	internal	
Owner:	VaultNote Team	
Confidentiality:	internal	(rated 2 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Build artifacts are the final deliverable of the supply chain. Integrity is critical — tampered artifacts compromise every environment they are deployed to.	
Processed by:	AWS ECR Container Registry, ECS Container Platform, GitHub Actions Build Pipeline, VaultNote Source Repository	
Stored by:	AWS ECR Container Registry, GitHub Actions Build Pipeline, VaultNote Source Repository	
Sent via:	Push to Pipeline, Push Image to Registry	
Received via:	Pull Images from ECR	
Data Breach:	**probable**	
Data Breach Risks:	This data asset has data breach potential because of 17 remaining risksStr:	

Probable: accidental-secret-leak@source-repo

Probable: code-backdooring@container-registry

Probable: code-backdooring@build-pipeline

Probable: code-backdooring@source-repo

Probable: missing-cloud-hardening@aws-cloud@aws

Probable: missing-cloud-hardening@external-cicd

Probable: path-traversal@ecs-platform@container-registry@ecs-platform>pull-images-from-ecr

Probable: path-traversal@build-pipeline@container-registry@build-pipeline>push-image-to-registry

Possible: server-side-request-forgery@ecs-platform@container-registry@ecs-platform>pull-images-from-ecr

Possible: server-side-request-forgery@build-pipeline@container-registry@build-pipeline>push-image-to-registry

Possible: server-side-request-forgery@source-repo@build-pipeline@source-repo>push-to-pipeline

Possible: unchecked-deployment@container-registry

Possible: unchecked-deployment@build-pipeline

Possible: unchecked-deployment@source-repo

Possible: unguarded-access-from-internet@container-registry@build-pipeline@build-pipeline>push-image-to-registry

Possible: unguarded-access-from-internet@build-pipeline@source-repo@source-repo>push-to-pipeline

Improbable: missing-hardening@ecs-platform

Embedding Vectors: 8 / 8 Risks

High-dimensional vector representations of user note content generated by the embedding service. Semantically encodes note information even without storing the original text.

ID:	embedding-vectors	
Usage:	business	
Quantity:	many	
Tags:	ai, embeddings	
Origin:	internal	
Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	operational	(rated 2 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Embedding vectors can be used in model inversion attacks to reconstruct approximate original text. Cross-user embedding access leaks semantic note content.	
Processed by:	LLM Note Summarizer, Note RAG Pipeline, VaultNote Vector Store	
Stored by:	VaultNote Vector Store	
Sent via:	Retrieve from Vector Store, Query Vector Store	
Received via:	none	
Data Breach:	**probable**	
Data Breach Risks:	This data asset has data breach potential because of 8 remaining risksStr:	

Probable: container-baseimage-backdooring@rag-pipeline

Probable: container-baseimage-backdooring@vector-store

Probable: container-platform-escape@ecs-platform

Possible: server-side-request-forgery@llm-summarizer@rag-pipeline@llm-summarizer>call-rag-pipeline

Possible: server-side-request-forgery@llm-summarizer@vector-store@llm-summarizer>query-vector-store

Possible: server-side-request-forgery@rag-pipeline@vector-store@rag-pipeline>retrieve-from-vector-store

Possible: unguarded-access-from-internet@rag-pipeline@llm-summarizer@llm-summarizer>call-rag-pipeline

Possible: unguarded-access-from-internet@vector-store@llm-summarizer@llm-summarizer>query-vector-store

File Attachments: 23 / 23 Risks

Binary file uploads attached to notes. May include documents, images, or other sensitive files depending on note usage.

ID:	file-attachments	
Usage:	business	
Quantity:	many	
Tags:	none	
Origin:	customer	
Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Files may contain sensitive documents. Integrity is critical because tampered attachments corrupt evidence or business records. MinIO's default credentials (minioadmin) make all stored files accessible to anyone on the network — intentional misconfiguration for demo purposes.	
Processed by:	API Server, AWS API Gateway, AWS S3 Notes Bucket, Browser SPA, MinIO Object Storage, Nginx Reverse Proxy	
Stored by:	AWS S3 Notes Bucket, MinIO Object Storage	
Sent via:	Route to ECS API, MinIO File Storage, HTTPS to Nginx, HTTP to API Server	
Received via:	Route to ECS API, MinIO File Storage, HTTPS to Nginx, HTTP to API Server	
Data Breach:	**probable**	
Data Breach Risks:	This data asset has data breach potential because of 23 remaining risksStr:	

Probable: container-baseimage-backdooring@api-server

Probable: container-baseimage-backdooring@minio-storage

Probable: container-baseimage-backdooring@nginx-proxy

Probable: container-platform-escape@ecs-platform

Probable: exposed-default-credentials@minio-storage

Probable: missing-cloud-hardening@aws-cloud@aws

Probable: missing-cloud-hardening@docker-host

Probable: path-traversal@api-server@minio-storage@api-server>minio-file-storage

Possible: lateral-movement-shared-runtime@docker-host

Possible: missing-authentication@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Possible: missing-authentication@aws-api-gateway>route-to-ecs-api@aws-api-gateway@api-server

Possible: missing-csp-header@nginx-proxy

Possible: missing-authentication-second-factor@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Possible: server-side-request-forgery@api-server@minio-storage@api-server>minio-file-storage

Possible: unencrypted-communication@api-server>minio-file-storage@api-server@minio-storage

Possible: unguarded-access-from-internet@api-server@aws-api-gateway@aws-api-gateway>route-to-ecs-api

Possible: unencrypted-communication@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Improbable: missing-waf@api-server

Improbable: unencrypted-asset@s3-notes-bucket

Improbable: unguarded-direct-datastore-access@api-server>minio-file-storage@api-server@minio-storage

Improbable: unnecessary-technical-asset@s3-notes-bucket

Improbable: mixed-targets-on-shared-runtime@docker-host

Improbable: unencrypted-asset@minio-storage

Note Content: 33 / 33 Risks

User-authored note text — may include personal, financial, or health information depending on how users employ the application.

ID:	note-content	
Usage:	business	
Quantity:	many	
Tags:	none	
Origin:	customer	
Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Notes may contain highly sensitive personal information. Integrity is critical because corrupted notes lose user trust.	
Processed by:	API Server, AWS API Gateway, AWS RDS PostgreSQL, Browser SPA, ECS Container Platform, LLM Note Summarizer, Nginx Reverse Proxy, Note RAG Pipeline, PostgreSQL Database, VaultNote Vector Store	
Stored by:	AWS RDS PostgreSQL, PostgreSQL Database	
Sent via:	Route to ECS API, PostgreSQL Connection, HTTPS to Nginx, HTTP to API Server, Call RAG Pipeline	
Received via:	Route to ECS API, Retrieve from Vector Store, Query Vector Store, PostgreSQL Connection, HTTPS to Nginx, HTTP to API Server, Call RAG Pipeline	
Data Breach:	**probable**	
Data Breach Risks:	This data asset has data breach potential because of 33 remaining risksStr:	

Probable: container-baseimage-backdooring@api-server

Probable: container-baseimage-backdooring@nginx-proxy

Probable: container-baseimage-backdooring@rag-pipeline

Probable: container-baseimage-backdooring@postgresql-db

Probable: container-baseimage-backdooring@vector-store

Probable: container-platform-escape@ecs-platform

Probable: missing-cloud-hardening@aws-cloud@aws

Probable: missing-cloud-hardening@docker-host

Probable: sql-nosql-injection@api-server@postgresql-db@api-server>postgresql-connection

Possible: lateral-movement-shared-runtime@docker-host

Possible: missing-authentication@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Possible: missing-authentication@aws-api-gateway>route-to-ecs-api@aws-api-gateway@api-server

Possible: missing-csp-header@nginx-proxy

Possible: missing-authentication-second-factor@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Possible: server-side-request-forgery@api-server@minio-storage@api-server>minio-file-storage

Possible: server-side-request-forgery@ecs-platform@container-registry@ecs-platform>pull-images-from-ecr

Possible: server-side-request-forgery@llm-summarizer@rag-pipeline@llm-summarizer>call-rag-pipeline

Possible: server-side-request-forgery@llm-summarizer@vector-store@llm-summarizer>query-vector-store

Possible: server-side-request-forgery@rag-pipeline@vector-store@rag-pipeline>retrieve-from-vector-store

Possible: unencrypted-communication@api-server>postgresql-connection@api-server@postgresql-db

Possible: unguarded-access-from-internet@api-server@aws-api-gateway@aws-api-gateway>route-to-ecs-api

Possible: unguarded-access-from-internet@rag-pipeline@llm-summarizer@llm-summarizer>call-rag-pipeline

Possible: unguarded-access-from-internet@vector-store@llm-summarizer@llm-summarizer>query-vector-store

Possible: unencrypted-communication@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Improbable: missing-hardening@rds-postgresql

Improbable: missing-hardening@ecs-platform

Improbable: missing-hardening@postgresql-db

Improbable: missing-waf@api-server

Improbable: unencrypted-asset@rds-postgresql

Improbable: unguarded-direct-datastore-access@api-server>postgresql-connection@api-server@postgresql-db

Improbable: unnecessary-technical-asset@rds-postgresql

Improbable: mixed-targets-on-shared-runtime@docker-host

Improbable: unencrypted-asset@postgresql-db

Notification Payloads: 3 / 3 Risks

Email notification payloads containing user email addresses and note titles sent via Lambda to AWS SES. Contains PII (email address) and indirect note content references.

ID:	notification-payloads	
Usage:	business	
Quantity:	many	
Tags:	pii	
Origin:	internal	
Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	operational	(rated 2 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Email addresses are PII. Note titles may be sensitive. Loss via DLQ absence creates unauditable data processing gaps.	
Processed by:	AWS SES, Lambda Email Notifier	
Stored by:	none	
Sent via:	SES Email Send	
Received via:	none	
Data Breach:	**probable**	
Data Breach Risks:	This data asset has data breach potential because of 3 remaining risksStr:	

Probable: missing-cloud-hardening@aws-cloud@aws

Possible: server-side-request-forgery@ecs-platform@container-registry@ecs-platform>pull-images-from-ecr

Possible: server-side-request-forgery@lambda-notifier@aws-ses@lambda-notifier>ses-email-send

Session Tokens: 16 / 16 Risks

JWT tokens and Redis session keys used to maintain authenticated state. Theft allows session hijacking without knowing the user's password.

ID:	session-tokens	
Usage:	business	
Quantity:	many	
Tags:	none	
Origin:	internal	
Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Session token compromise enables full session hijacking. Short TTLs and revocation lists mitigate but do not eliminate the risk.	
Processed by:	API Server, Redis Cache	
Stored by:	Redis Cache	
Sent via:	Redis Session Store	
Received via:	Redis Session Store	
Data Breach:	**probable**	
Data Breach Risks:	This data asset has data breach potential because of 16 remaining risksStr:	

Probable: container-baseimage-backdooring@api-server

Probable: container-baseimage-backdooring@redis-cache

Probable: container-platform-escape@ecs-platform

Probable: missing-cloud-hardening@docker-host

Probable: sql-nosql-injection@api-server@redis-cache@api-server>redis-session-store

Possible: missing-authentication@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Possible: missing-authentication@aws-api-gateway>route-to-ecs-api@aws-api-gateway@api-server

Possible: missing-authentication-second-factor@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Possible: server-side-request-forgery@api-server@minio-storage@api-server>minio-file-storage

Possible: unencrypted-communication@api-server>redis-session-store@api-server@redis-cache

Possible: unguarded-access-from-internet@api-server@aws-api-gateway@aws-api-gateway>route-to-ecs-api

Possible: unencrypted-communication@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Improbable: missing-waf@api-server

Improbable: unencrypted-asset@redis-cache

Improbable: unguarded-direct-datastore-access@api-server>redis-session-store@api-server@redis-cache

Improbable: mixed-targets-on-shared-runtime@docker-host

User Credentials: 26 / 26 Risks

Email addresses and bcrypt-hashed passwords used for authentication. Compromise allows full account takeover.

ID:	user-credentials	
Usage:	business	
Quantity:	many	
Tags:	none	
Origin:	customer	
Owner:	VaultNote Team	
Confidentiality:	strictly-confidential	(rated 5 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Credentials grant full account access. A breach leaks all users' identities and enables account takeover across every note and attachment.	
Processed by:	API Server, AWS API Gateway, AWS RDS PostgreSQL, Browser SPA, ECS Container Platform, Nginx Reverse Proxy, PostgreSQL Database	
Stored by:	AWS RDS PostgreSQL, PostgreSQL Database	
Sent via:	Route to ECS API, PostgreSQL Connection, HTTPS to Nginx, HTTP to API Server	
Received via:	PostgreSQL Connection	
Data Breach:	**probable**	
Data Breach Risks:	This data asset has data breach potential because of 26 remaining risksStr:	

Probable: container-baseimage-backdooring@api-server

Probable: container-baseimage-backdooring@nginx-proxy

Probable: container-baseimage-backdooring@postgresql-db

Probable: container-platform-escape@ecs-platform

Probable: missing-cloud-hardening@aws-cloud@aws

Probable: missing-cloud-hardening@docker-host

Probable: sql-nosql-injection@api-server@postgresql-db@api-server>postgresql-connection

Possible: lateral-movement-shared-runtime@docker-host

Possible: missing-authentication@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Possible: missing-authentication@aws-api-gateway>route-to-ecs-api@aws-api-gateway@api-server

Possible: missing-csp-header@nginx-proxy

Possible: missing-authentication-second-factor@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Possible: server-side-request-forgery@api-server@minio-storage@api-server>minio-file-storage

Possible: server-side-request-forgery@ecs-platform@container-registry@ecs-platform>pull-images-from-ecr

Possible: unencrypted-communication@api-server>postgresql-connection@api-server@postgresql-db

Possible: unguarded-access-from-internet@api-server@aws-api-gateway@aws-api-gateway>route-to-ecs-api

Possible: unencrypted-communication@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Improbable: missing-hardening@rds-postgresql

Improbable: missing-hardening@ecs-platform

Improbable: missing-hardening@postgresql-db

Improbable: missing-waf@api-server

Improbable: unencrypted-asset@rds-postgresql

Improbable: unguarded-direct-datastore-access@api-server>postgresql-connection@api-server@postgresql-db

Improbable: unnecessary-technical-asset@rds-postgresql

Improbable: mixed-targets-on-shared-runtime@docker-host

Improbable: unencrypted-asset@postgresql-db

AI Model Responses: 2 / 2 Risks

LLM-generated summaries and explanations of user notes. May contain paraphrased versions of sensitive note content. Cached temporarily in the API response layer.

ID:	ai-model-responses	
Usage:	business	
Quantity:	many	
Tags:	ai	
Origin:	internal	
Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	operational	(rated 2 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	LLM responses summarise user note content. If the model was manipulated via prompt injection, responses may leak or alter content from other notes.	
Processed by:	LLM Note Summarizer	
Stored by:	none	
Sent via:	none	
Received via:	none	
Data Breach:	**possible**	
Data Breach Risks:	This data asset has data breach potential because of 2 remaining risksStr:	

Possible: server-side-request-forgery@llm-summarizer@rag-pipeline@llm-summarizer>call-rag-pipeline

Possible: server-side-request-forgery@llm-summarizer@vector-store@llm-summarizer>query-vector-store

Trust Boundaries

In total **7 trust boundaries** have been modeled during the threat modeling process.

AI Services

Internal network housing the AI/ML components (vector store, RAG pipeline). The LLM inference endpoint is internet-accessible — it is in the Internet trust boundary. AI components have outbound access to the vector store and receive note content from the API server.

ID:	ai-services
Type:	network-on-prem
Tags:	ai
Assets inside:	Note RAG Pipeline, VaultNote Vector Store
Boundaries nested:	none

AWS Cloud

AWS production environment (VPC). Contains managed services (RDS, S3, ECS, Lambda, API Gateway). Separated from the dev Docker environment by an air gap — production traffic flows through the API Gateway, never through the Docker compose setup.

ID:	aws-cloud
Type:	network-cloud-provider
Tags:	aws, cloud-native
Assets inside:	AWS API Gateway, AWS SES, ECS Container Platform, Lambda Email Notifier, AWS RDS PostgreSQL, AWS S3 Notes Bucket
Boundaries nested:	none

Application Network

Internal Docker bridge network (frontend-net). Contains the API server which is reachable from Nginx but also bridges into backend-net.

ID:	application-network
Type:	network-on-prem
Tags:	none
Assets inside:	API Server
Boundaries nested:	none

DMZ

Demilitarized zone containing the Nginx reverse proxy. Accessible from the internet but isolated from the application and data tiers.

ID:	dmz
Type:	network-on-prem
Tags:	none
Assets inside:	Nginx Reverse Proxy
Boundaries nested:	none

Data Tier

Internal Docker bridge network (backend-net, marked internal: true). No external routing. Contains all datastores. The API server has a NIC on this network and acts as the sole legitimate access path.

ID:	data-tier
Type:	network-on-prem
Tags:	none
Assets inside:	MinIO Object Storage, PostgreSQL Database, Redis Cache
Boundaries nested:	none

External CI/CD

GitHub-hosted CI/CD infrastructure and AWS ECR registry. External to the application runtime but directly connected to it via image pull at deploy time. Trust boundary represents the entire supply chain attack surface.

ID:	external-cicd
Type:	network-cloud-provider
Tags:	supply-chain
Assets inside:	GitHub Actions Build Pipeline, AWS ECR Container Registry, VaultNote Source Repository
Boundaries nested:	none

Internet

Untrusted external network. All traffic from this zone must cross the Nginx TLS termination point before entering the DMZ.

ID:	internet

Type:	network-on-prem
Tags:	none
Assets inside:	Browser SPA
Boundaries nested:	none

Shared Runtimes

In total **1 shared runtime** has been modeled during the threat modeling process.

Docker Host
All containers run on the same Docker host. A container escape or privileged container could access the host filesystem including Docker socket and database volumes.

ID:	docker-host
Tags:	container-runtime, docker
Assets running:	Nginx Reverse Proxy, API Server, PostgreSQL Database, Redis Cache, MinIO Object Storage

Risk Rules Checked by Threagile

Threagile Version: 1.0.0
Threagile Build Timestamp:
Threagile Execution Timestamp: 20260604113007
Model Filename: threagile.yaml
Model Hash (SHA256): 15d123b13753140c59d8c6698da83a704aebcacf211732f0ee2ef3b5931b60ef

Threagile (see https://threagile.io for more details) is an open-source toolkit for agile threat modeling, created by Christian Schneider (https://christian-schneider.net): It allows to model an architecture with its assets in an agile fashion as a YAML file directly inside the IDE. Upon execution of the Threagile toolkit all standard risk rules (as well as individual custom rules if present) are checked against the architecture model. At the time the Threagile toolkit was executed on the model input file the following risk rules were checked:

Disclaimer

VaultNote Security Team conducted this threat analysis using the open-source Threagile toolkit on the applications and systems that were modeled as of this report's date. Information security threats are continually changing, with new vulnerabilities discovered on a daily basis, and no application can ever be 100% secure no matter how much threat modeling is conducted. It is recommended to execute threat modeling and also penetration testing on a regular basis (for example yearly) to ensure a high ongoing level of security and constantly check for new attack vectors.

This report cannot and does not protect against personal or business loss as the result of use of the applications or systems described. VaultNote Security Team and the Threagile toolkit offers no warranties, representations or legal certifications concerning the applications or systems it tests. All software includes defects: nothing in this document is intended to represent or warrant that threat modeling was complete and without error, nor does this document represent or warrant that the architecture analyzed is suitable to task, free of other defects than reported, fully compliant with any industry standards, or fully compatible with any operating system, hardware, or other application. Threat modeling tries to analyze the modeled architecture without having access to a real working system and thus cannot and does not test the implementation for defects and vulnerabilities. These kinds of checks would only be possible with a separate code review and penetration test against a working system and not via a threat model.

By using the resulting information you agree that VaultNote Security Team and the Threagile toolkit shall be held harmless in any event.

This report is confidential and intended for internal, confidential use by the client. The recipient is obligated to ensure the highly confidential contents are kept secret. The recipient assumes responsibility for further distribution of this document.

In this particular project, a time box approach was used to define the analysis effort. This means that the author allotted a prearranged amount of time to identify and document threats. Because of this, there is no guarantee that all possible threats and risks are discovered. Furthermore, the analysis applies to a snapshot of the current state of the modeled architecture (based on the architecture information provided by the customer) at the examination time.

Report Distribution

Distribution of this report (in full or in part like diagrams or risk findings) requires that this disclaimer as well as the chapter about the Threagile toolkit and method used is kept intact as part of the distributed report or referenced from the distributed parts.